

2020 - 2021

PROJET DE FIN D'ÉTUDES

DIPLÔME NATIONAL D'INGÉNIEUR

INFORMATIQUE
SPÉCIALITÉ : CLOUD COMPUTING

**Mise en place d'un processus automatique
d'intégration DevOps**

Réalisé par : TOUJ BELHASSEN

Encadrant Académique : Mr. ANOUAR CHEMEK

Encadrant Professionnel : Mr. ABDESLEM DRIRA

Entreprise d'accueil :



Dédicace

*A Dieu source de toute connaissance
A ma raison de vivre, d'espérer,
A ma source de courage, à ceux que j'ai de plus
cher*

Ma mère, mon père

Mes frères

Mes sœurs

Pour leur amour, leur confiance

Et leur sacrifice sans limite

A tous mes professeurs

Qui m'ont soutenu

De près ou de loin tout au long de ce projet.

A tous mes amis

A toute personne

Qui m'a aidé à franchir un horizon dans ma vie...

BELHASSEN



Remerciements

Ce travail n'aurait pas pu être réalisé sans le soutien et l'aide de plusieurs personnes et institutions.

J'adresse mes sincères remerciements :

A mon encadrant universitaire Mr. Anouar Chemek pour son encadrement, ses remarques pertinentes et ses recommandations fortes enrichissantes dont j'ai bénéficié tout au long de ce stage, qu'il trouve, ainsi, mes sentiments les plus respectueux.

Je remercie et j'exprime ma gratitude envers Mr Abdeslem Drira mon encadreur à l'entreprise **STUDI**, pour avoir accepté de diriger ce projet, sa disponibilité et ses conseils qui m'ont été très bénéfiques.

Je souhaite exprimer ma gratitude à l'entreprise **STUDI** qui m'a accueilli et m'a offert un cadre de travail idéal. En effet, j'ai eu le plaisir de travailler dans une entreprise de grande allure, reconnue à l'échelle mondiale.

Mes remerciements vont à l'ensemble de l'équipe de **STUDI** pour leur accueil chaleureux, pour l'expérience enrichissante et pleine d'intérêt qu'ils m'ont fait vivre durant ces quatre mois.

J'exprime mes sentiments de gratitude à tous mes professeurs qui ont participé en évolution scientifique durant les quatre années écoulées de formation à **l'ESPRIT**.

Table des matières

Introduction Générale	1
Chapitre 1 : Contexte Général	2
Introduction.....	3
I. Présentation Générale de l'entreprise	3
1. Activités principales	3
2. Organisation de la société.....	4
II. Etude de l'existant.....	4
III. Problématique	5
IV. Solution proposée	6
V. Méthodologie de travail	7
VI. Méthode choisie	8
Conclusion	9
Chapitre 2 : Étude théorique	10
Introduction.....	11
I. Concepts de base	11
1. Conteneurisation.....	11
2. Dev/Ops.....	12
3. CI/DI.....	13
4. Monitoring et alerting.....	14
5. Micro services	15
6. Infrastructure as a code (IAAC)	16
7. Plateforme de gestion de conteneur.....	17
8. Gestion de code source	17
9. Automatisation et gestion de configuration	18
10. Orchestration de containers	18
II. Etude comparative des technologies adoptées	19

1. Outils de gestion de versions.....	19
2. Outils de gestion de build et d'automatisation.....	20
3. Outils de monitoring.....	21
4. Outils d'orchestration de containers.....	23
5. Solution de conteneurisation adoptée.....	24
6. Les outils d'automatisation et gestion de configuration.....	25
Conclusion	26
Chapitre 3 : Analyse des besoins et Conception.....	28
Introduction.....	29
I. Analyse des besoins	29
1. Les besoins fonctionnels	29
2. Les besoins non fonctionnels	30
3. Identification des acteurs.....	30
II. Représentation des besoins	31
1. Cas d'utilisation général.....	31
2. Raffinement des cas d'utilisation	33
a. Cas d'utilisation « Gérer la liste des images docker ».....	33
b. Cas d'utilisation « Gestion des profils ».....	36
c. Cas d'utilisation « superviser l'état du plateforme ».....	37
d. Cas d'utilisation « Générer un nouveau projet »	39
3. Diagramme séquence système « s'authentifier »	41
4. Diagramme séquence système « Supprimer une image docker »	42
III. Conception	43
1. Architecture générale de la solution.....	43
2. Diagramme de déploiement	45
3. Aspect dynamique de la solution	46
a. Diagramme d'activité « Authentification »	46

b. Diagramme d'activité « Gérer la liste des images docker »	47
Conclusion	48
Chapitre 4 : Réalisation.....	49
Introduction.....	50
I. Environnement de travail	50
1. Environnement matériel	50
2. Environnement logiciel	50
3. Choix Technologiques.....	50
II. Principaux développements	52
1. Génération d'un nouveau Pipeline	52
2. Intégration et Déploiement Continue	54
a. Intégration Continue	55
b. Déploiement Continu.....	58
3. Supervision des conteneurs	61
Conclusion	64
Conclusion générale et perspectives	65
Webographie	66
Liste des acronymes.....	67
Résumé.....	68

Liste des figures

Figure 1 : l'approche de la méthode Scrum ¹	9
Figure 2: Conteneur.....	11
Figure 3: Cycle de vie DevOps	12
Figure 4 : Concept CI/CD	13
Figure 5 : Schéma de fonctionnement du monitoring	15
Figure 6 : Architecture monolithique vs Micro services.....	16
Figure 7 : Principe de gestion de versionning	17
Figure 8 : Diagramme de cas d'utilisation générale.....	32
Figure 9 : Diagramme de cas d'utilisation « Gérer la liste des images docker »	34
Figure 10 : Diagramme de cas d'utilisation « Gestion des profils »	36
Figure 11: Diagramme de cas d'utilisation « superviser l'état du plateforme »	38
Figure 12 : Diagramme de cas d'utilisation « Générer un nouveau projet ».....	40
Figure 13 : Diagramme séquence système « s'authentifier ».....	42
Figure 14 : Diagramme séquence système « Supprimer une image docker ».....	43
Figure 15 : Architecture générale de la solution	44
Figure 16 : Architecture générale de la solution	45
Figure 17 : Diagramme de déploiement	45
Figure 18 : Diagramme d'activité « Authentification »	47
Figure 19: Diagramme d'activité « Gérer la liste des images docker »	48
Figure 20 : capture écran montrant les jobs dans Jenkins	52
Figure 21 : Les étapes à faire pour générer un Job.....	53
Figure 22 : Rapport du build et le Stage View du pipeline	54
Figure 23 : Le projet implémenté sur Gitlab	55
Figure 24 : Changement du code source manuellement.....	56
Figure 25 : Déclenchement d'un build automatiquement.....	56
Figure 26 : Déclenchement d'un test avec JUnit	57
Figure 27 : Déclenchement d'un test Qualité avec SonarQube	57
Figure 28 : SonarQube Dashboard	58
Figure 29 : Déclenchement d'un Build d'image docker	59
Figure 30 : Liste des images pousser dans nexus	59
Figure 31 : Résultat d'exécution de Playbook d'ansible.....	60
Figure 32 : Affichage des pod avec Kubernetes	60

Figure 33 : L'interface de l'application déployée	61
Figure 34 : Prometheus Target	61
Figure 35 : Dashboard Kubernetes cluster metrics	62
Figure 36 : Dashboard Docker metrics.....	63
Figure 37 : Dashboard Jenkins Performance.....	63

Liste des tableaux

Tableau 1 : Organisation du groupe Studi.....	4
Tableau 2 : comparatif des méthodologies.....	8
Tableau 3 : Tableau comparatif Github vs Gitlab	20
Tableau 4 : Tableau comparatif Jenkins vs Bamboo vs TravisCI.....	21
Tableau 5 : Tableau comparatif Prometheus vs Grafana vs Kibana vs Nagios	22
Tableau 6 : Tableau comparatif Kubernetes vs Docker Swarm.....	24
Tableau 7 : Tableau comparatif Docker vs Rocket vs LXC/LXD	25
Tableau 8 : Tableau comparatif Ansible vs Puppet vs Chef	26
Tableau 9 : Description Textuelle du cas d'utilisation « Gérer la liste des images docker » ...	35
Tableau 10 : Description Textuelle du cas d'utilisation « Gérer les profils utilisateurs »	37
Tableau 11 : Description Textuelle du cas d'utilisation « superviser l'état du plateforme » ...	39
Tableau 12 : Description Textuelle du cas d'utilisation « Générer un nouveau projet ».....	41
Tableau 13 : Technologies utilisés	52

Introduction Générale

Nous assistons aujourd'hui à une course acharnée dans le domaine informatique pour satisfaire les nouvelles attentes des équipes opérationnelles et des développeurs.

En effet, afin d'automatiser les opérations aux équipes opérationnelles et les développeurs, la culture *DevOps* s'avère apporter de meilleures réponses aux entreprises qui cherchent à simplifier le travail des employés. Cette nouvelle tendance ne cesse de s'accroître et de se développer.

C'est dans ce cadre de réflexion que nous avons décidé de choisir ce projet afin de le mettre à disposition aux équipes de développement de la société STUDI International afin d'automatiser les processus. La culture *DevOps* offre ainsi une mise en production plus rapide et une meilleure qualité grâce à ces outils essentiellement la conteneurisation et les microservices.

Notre étude se subdivise en cinq chapitres. Le premier est consacré à une présentation du contexte général où nous exposerons le périmètre du projet, de l'entreprise d'accueil et ensuite nous expliquerons le choix de la méthodologie de travail.

Dans le deuxième chapitre "Etat de l'art" nous présenterons les principaux concepts autour desquels le projet évolue. Le troisième chapitre est une "Analyse des besoins et solutions techniques", des besoins fonctionnels et non fonctionnels ainsi qu'une présentation des différents choix des outils techniques appropriés.

La conception de notre application sera présentée dans le quatrième chapitre "Architecture générale et Conception" et nous consacrerons le dernier chapitre « Réalisation » au développement de notre outil.

Chapitre 1 : Contexte Général

Introduction

Dans le présent chapitre, nous présentons le cadre général du projet, en premier lieu nous commençons par une présentation de l'organisme d'accueil, ensuite nous exposons le problème et la solution à atteindre. Enfin, nous présentons la proposée et la méthodologie que nous avons adoptée dans le cadre de ce projet.

I. Présentation Générale de l'entreprise

Notre organisme d'accueil est la société internationale STUDI considérée comme une des importantes entreprises d'ingénierie en Afrique. La société STUDI International est active dans plus de trente pays et elle a des filiales dans plus de dix pays Africains, et en France. Elle gère ses activités à partir de son siège à Tunis.

En effet, les activités de la société STUDI International sont plus centrées sur l'Afrique, ces projets influencent surtout le développement durable économique et social. Le groupe STUDI International gère tous les stades de réalisation d'un projet, depuis la phase d'identification jusqu'à l'assistance du client dans le pilotage et la réception des travaux. Les projets sont en effet des contrats signés avec les Etats dont le financement est essentiellement assuré soit par les États ou des fonds dédiés au développement du continent Africain (BM, BAD, AFD, KFAED, FADES, etc.).

1. Activités principales

Parmi les activités principales de la société STUDI nous citons :

- Infrastructures de Transport et Aménagements Côtiers
- Construction et Industrie
- Eau, Agriculture et Environnement
- Énergies Conventionnelles et Renouvelables / Pétrole et Gaz
- Architecture et Urbanisme
- Ordonnancement, Pilotage et Coordination

Toutefois les activités de la société ne sont pas limitées à la liste mentionnée ci-dessus en effet, des filiales complémentaires compte parmi les activités de la société notamment le domaine informatique. Ainsi les technologies de l’informatiques permettent à la société d’effectuer des tâches touchant le développement et l’intégration de solutions informatiques et offrent conseil et assistance à la maîtrise d’ouvrage dans la mise en place de réseaux de communication et de solutions informatiques.

2. Organisation de la société

	Technologie de l’information Système d’information géographique
	Etudes stratégiques et institutionnelles Études économiques et environnementales
	Mécanique des sols et des matériaux
	Conseil financier

Tableau 1 : Organisation du groupe Studi

II. Etude de l’existant

La société en question utilise une infrastructure virtualisée basée sur la solution VMWARE qui permet de fournir des machines virtuelles depuis des serveurs physiques.

Une machine virtuelle spécifique permet l’exécution d’une application à partir d’un système d’exploitation et des ressources en termes CPU et RAM propre à cette machine. Toutefois, il faut prendre en considération qu’une application n’exploite pas le plus souvent toutes les ressources consacrées à une VM.

En effet, l'implémentation d'une application nécessite l'utilisation de plusieurs VM et l'enchaînement de nombreuses étapes avant d'arriver à l'étape de production, ainsi toutes les données sont stockées dans un seul répertoire central, ce qui peut entraîner une perte de données si ce dernier est endommagé.

Ainsi, nous pouvons constater que le recours au VM présentent des inconvénients ce qui nous a incité à chercher et exposer ces derniers dans la partie suivante.

III. Problématique

Dans cette partie nous présentons les différents éléments qui nous ont incité à penser à ce projet. Ces éléments sont les différents problèmes rencontrés par les développeurs et les administrateurs système pour la mise en production des applications.

Parmi ces problèmes, nous citons :

- Les différents problèmes de compatibilité et d'adaptabilité.
 - Ce problème est le résultat du changement de l'environnement d'exécution.
- Les tâches effectuées par les opérateurs sont répétitives et manuelles.
 - La configuration du serveur n'est pas automatisée, ce qui peut provoquer des difficultés. Ces complications dû à une baisse de rendement, de rapidité et du temps.
- Les sauvegardes et la restauration des applications sont obligatoirement effectuées essentiellement avec l'OS, et ses fichiers systèmes.
 - Problème d'espace de stockage disponible pour le backup.
 - Problème de lenteur de sauvegarde et de restauration
- La difficulté d'analyser les logs systèmes pour détecter l'origine du problème.
- Le test du code dans un environnement au cours de production, nous devons obligatoirement passer une demande aux équipes opérationnelles pour le tester ce qui peut provoquer des pertes du temps.
 - Les développeurs ne sont pas autonomes.

- La lourdeur de mise en œuvre et de gestion des VM.

Ces différents problèmes signalés ont incité le groupe STUDI à chercher une solution qui peut rendre le Système d'Information plus performant, plus agile et moins coûteux.

IV. Solution proposée

Pour rectifier les problèmes cités dans la partie précédente, le Groupe STUDI nous a proposé de mettre en place une infrastructure basée sur des produits open source pour héberger tous les services.

En effet, le projet consiste à implémenter la conteneurisation qui nous offrira une virtualisation des ressources de manière légère, flexible et puissante.

La conteneurisation est une technique qui permet une nouvelle gestion des méthodes dont nous développons, gérons et déployons les applications.

Ainsi avec l'utilisation d'autres produits open source, le développement des applications va être plus rapide et nous pouvons bénéficier d'une infrastructure automatisée avec des tests et surveillance continue.

Cette nouvelle infrastructure doit s'articuler autour des caractéristiques suivantes :

- Réduction des coûts de développement et d'exploitation.
- Application conteneurisée indépendante.
- Cycle de développement plus court (grâce à l'automatisation)
- Déploiement rapide des nouveaux environnements.
- Supervision totale et continue de la plateforme.
- Une plateforme qui peut gérer les micros-services.
- La haute disponibilité applicative.
- Gestion plus rapide d'implémentation d'une nouvelle plateforme.

V. Méthodologie de travail

Notre objectif, dans le cadre de ce projet, est de bénéficier d'un contrat SLA garantissant financièrement un temps de disponibilité pour éviter tout impacte externe. C'est pour cette raison que nous devons suivre une démarche établie afin de bien gérer le projet.

À cet égard, il existe plusieurs méthodes de travail qui nous garantissent l'achèvement de l'application à temps précis.

Nous présentons ci-dessous quelques méthodes de gestion de projet :

- Cascade : est désignée comme une méthode de gestion de projet traditionnelle, consiste à suivre une succession d'étapes prédéfinies de façon séquentielle (Exigences [analyse des besoins du donneur d'ordre] à Conception à Planning [échéancier et budget] à Mise en œuvre à Vérification à Maintenance)
- Les méthodes Agiles : s'oppose généralement à la méthode traditionnelle classique. Elle est plus souple et adaptée, et place les besoins du client au centre des priorités du projet avec sa validation pour chaque sprint.
- La méthode du chemin critique : cette méthode nous permet de connaître une estimation claire du temps nécessaire pour accomplir chaque tâche, ainsi que la meilleure solution de l'ensemble des activités à effectuer pour que le projet soit achevé à la date définie.
- Scrum : elle suit les principes de la méthodologie agile dans lequel le projet se divise en plusieurs sprints, en plus le Scrum se compose de plusieurs éléments fondamentaux (des rôles, des événements, des artefacts, des règles.).

Nous présentons ci-dessous un tableau comparatif entre des différentes méthodes de travail :

Méthode	Approche	Relation client	Planification	Développement
Cascade	Prédictive	Interaction au besoin	Complète en amont	Longs cycles
Agile	Adaptive	Participation-Propriétaire de produit	Minimale	Courtes itération
Chemin critique	Prédictive	Interaction au besoin	Complète avec complexité	Longs cycles
Scrum	Adaptive	Participation-Propriétaire de produit	Minimale	Courtes itération

Tableau 2 : comparatif des méthodologies

VI. Méthode choisie

Pour bien gérer la production des applications et garantir la qualité de métier et suite à la recherche d'une méthode adéquate et convenante, nous avons privilégié que le Groupe Studi utilise la méthode Scrum qui est la plus éprouvée et la plus documentée.

En effet, l'approche Scrum suit les principes de la méthodologie Agile, autrement-dit l'implication et la participation active du client tout au long du projet.

Cette approche est divisée par plusieurs équipe auto-organisée et pluridisciplinaire qui se compose de :

- **Un Scrum Master**
- **Un Product Owner**
- **Une équipe de développement**

La figure suivante représente l'approche de cette méthode Scrum :



Figure 1 : l'approche de la méthode Scrum ¹

Plusieurs avantages peuvent être obtenues, nous pouvons citer :

- Une meilleure gestion du rendement de l'investissement
- Une plus grande autonomie des développeurs
- Une meilleure collaboration entre les équipes
- Une meilleure gestion globale des risques
- Une meilleure réactivité pour ajuster le produit aux réalités du marché

Conclusion

Dans le cadre de ce chapitre nous avons présenté un aperçu global de notre projet (présentation de l'entreprise, étude de l'existant, problématique, solution proposée et choix de méthodologie de travail), dans ce qui suit nous entamerons le chapitre étude théorique.

Chapitre 2 : Étude théorique

Introduction

Dans ce présent chapitre nous commençons par définir les différents concepts de base de la culture DevOps et ensuite nous effectuons une étude comparative des différentes technologies adoptées afin de mieux cerner les outils à utiliser.

I. Concepts de base

Cette section présente tous les concepts et les notions de base en relation avec la culture DevOps et qui aident à mieux cerner l'objectif de notre projet.

1. Conteneurisation

La mise au point de la méthode de la conteneurisation est considérée comme une révolution dans le domaine informatique. Cette méthode permet aux développeurs d'améliorer la productivité vu que les conteneurs transforment la façon de concevoir les applications.

En effet, la conteneurisation est une méthode permettant d'exécuter et d'isoler une application dans un environnement virtuel avec la totalité des éléments nécessaires pour fonctionner dans une seule zone appelée Conteneur, comme l'indique la figure 2 :

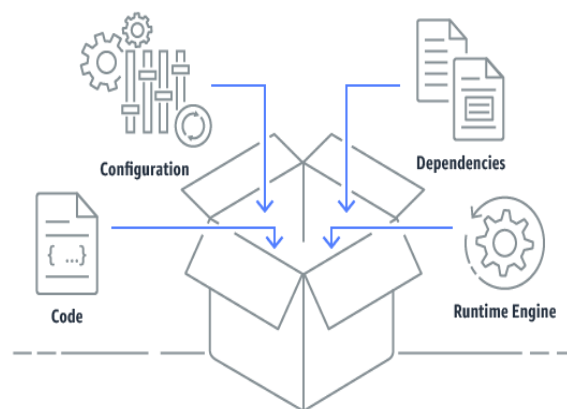


Figure 2: Conteneur

Ce conteneur se connecte directement au noyau d'un système, par conséquent il n'est pas obligatoire d'installer un nouveau système d'exploitation, comme pour les machines virtuelles vu que le noyau gère les ressources de l'ordinateur et permet la communication entre les différents composants matériels et logiciels.

Les différents avantages de la méthode de conteneurisation sont les suivantes² :

- La flexibilité : la conteneurisation gère même les applications les plus complexes.
- La légèreté : le noyau hôte est exploité et partagé par les conteneurs.
- Interchangeabilité : le déploiement des mises à jour est à la volée
- Evolutivité : les répliques (les clones) de conteneur peuvent être augmentés et distribués automatiquement.

2. Dev/Ops

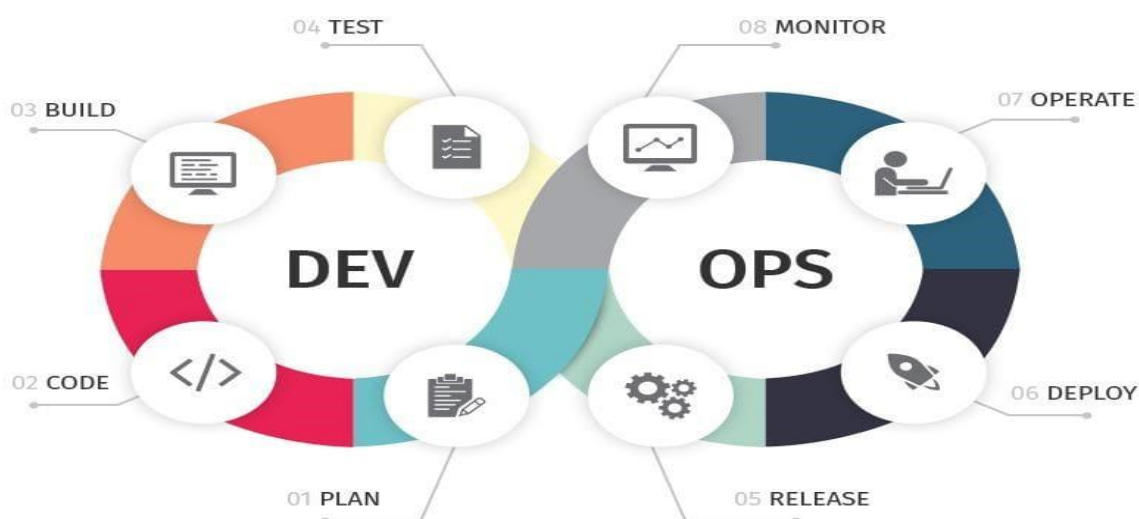


Figure 3: Cycle de vie DevOps

Le mot DevOps est étymologiquement constitué de l'abréviation des deux mots de l'anglais dev (development) et ops (operations). En effet, ce terme est usité pour définir l'ensemble des fonctions effectuées par une équipe de développement des applications (Dev) ainsi que l'exploitation des systèmes (Ops). Cette combinaison de tâches permet à l'entreprise de produire des applications et des services à un rythme accéléré et mieux servir leurs clients,

par rapport aux entreprises qui adoptent les processus traditionnels de développement des logiciels ainsi que la gestion de l'infrastructure.

3. CI/DI

Dans le domaine du génie logiciel 'software engineering', le CI/CD réfère généralement aux concepts de l'intégration continue et la distribution continue et/ou le déploiement continu. Autrement dit cette expression peut correspondre seulement aux deux concepts l'intégration continue du 'CI' et la distribution continue du 'CD', ou elle peut correspondre aux trois pratiques notamment l'intégration continue, la distribution continue et le déploiement continu. Cependant l'expression même 'distribution continue' peut inclure le processus de déploiement continu.

Cette approche garantit d'une part l'automatisation des actions de déploiement des applications ce qui permet d'augmenter la fréquence de distributions des applications, et d'autre part la surveillance continue tout au long des différentes phases de vie des applications (intégration continue, livraison continue et déploiement continu)

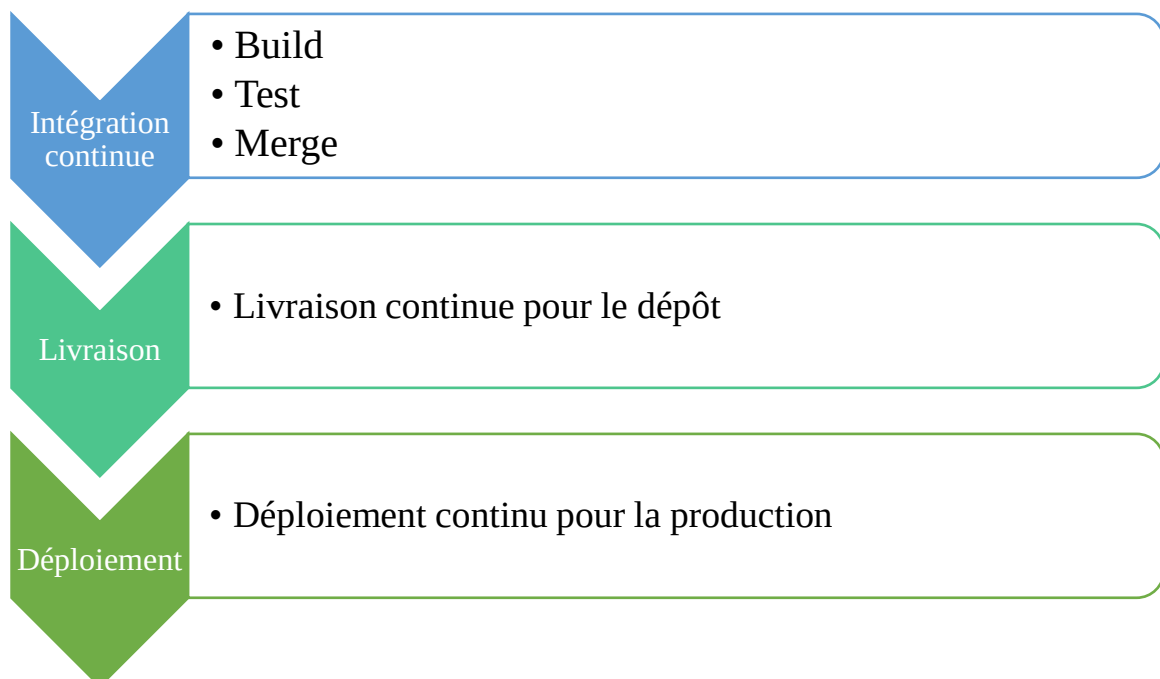


Figure 4 : Concept CI/CD

L'intégration continue (CI), permet aux développeurs de fusionner le plus souvent leurs changements de code vers la branche principale. Une fois les modifications sont validées l'application est créée automatiquement (**Build**) et des tests automatisés (**Test**), la clé de l'intégration continue, sont exécutés pour vérifier l'exactitude de la modification. Les dysfonctionnements, en cas d'un conflit entre le code existant et le nouveau, sont détectés rapidement au niveau du cycle de développement afin de les résoudre plus rapidement et plus fréquemment.

Après l'intégration continue, la livraison continue qui est en effet une extension de CI, automatise la publication du code validé. La version prête pour la publication peut être mis pour la production automatiquement.

Finalement, le déploiement continu va compléter le processus de livraison continue en partageant automatiquement le déploiement des applications au client à chaque changement. Ceci est considéré comme un grand avantage pour la qualité logicielle, les délais de projet et le coût.

4. Monitoring et alerting

Monitoring un terme anglais qu'en français signifie une activité de surveillance et de supervision. En effet, cette activité vous donne un aperçu de l'état des applications et l'infrastructure afin de dépister et de comprendre les éventuels problèmes. Ainsi, avoir un bon système de surveillance vous permettra de collecter et visualiser les données et enfin alerter les opérateurs en cas de problème.

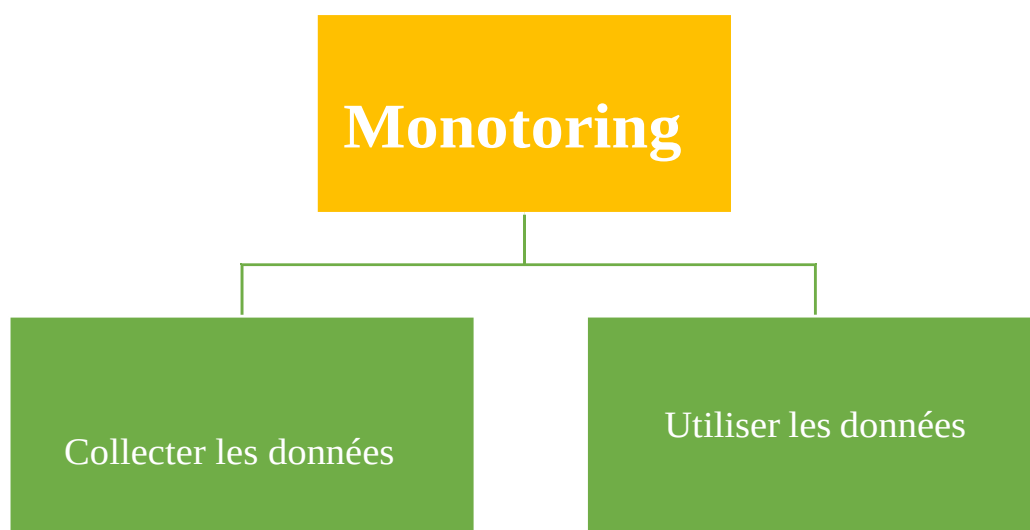


Figure 5 : Schéma de fonctionnement du monitoring

Le premier processus de l'action Monitoring est la collecte des données dans un système de surveillance, responsable de stockage, qui offre une visibilité sur l'état de vos systèmes et de vos tâches afin de collecter l'impact des modifications effectuées. Cependant il faut choisir les bons métriques ³.

Le deuxième processus est l'utilisation des données collectées. En effet, après avoir visualisées les données il faut les rendre accessibles afin de prendre des décisions pertinentes rapidement.

Le terme alerting signifie en français alerter, en effet comme nous l'avons déjà présenté, l'action monitoring permet d'analyser la situation du système, et c'est grâce à la fonction alerting qu'on peut intervenir pour exécuter des actions.

Toutefois, la distinction entre les deux actions monitoring et alerting n'est pas toujours prise en compte. En effet, monitoring est le plus souvent utilisé pour désigner monitoring et alerting en même temps ⁴.

5. Micro services

Avant l'apparition de l'architecture micro-service, les applications sont construites de manière monolithique, qui est considérée aujourd'hui comme méthode impertinente, qui n'est

plus judicieuse. L'utilisation de l'architecture Monolithique, rend l'application très volumineuse avec des difficultés à enrichir les fonctions et le traitement des problèmes.

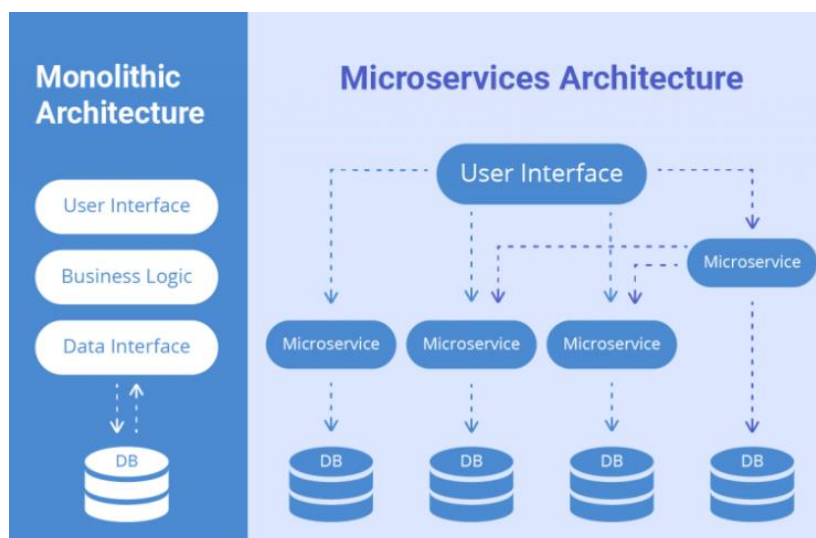


Figure 6 : Architecture monolithique vs Micro services

Comme l'illustre la figure 6, l'architecture Micro-service est formée pour décomposer les applications en des éléments autonomes connectés les uns aux autres avec API REST, toutefois l'approche monolithique est composée d'une entité indissociable qui forme tous les composants.

Les Micro-services conformes avec la culture DevOps favorisent les conteneurs qui eux-mêmes fonctionnent avec des composants indépendants, et mis à part de leurs environnements.

6. Infrastructure as a code (IAAC)

L'infrastructure en tant que code est une pratique qui permet aux administrateurs système la gestion et l'automatisation de l'infrastructure grâce à des scripts de programmation sans passer par des processus manuels, on l'appelle aussi l'infrastructure programmable.

Les administrateurs système implémentent une infrastructure qui offre des environnements stables, ainsi qu'avec une demande définit, on peut déployer rapidement un environnement cible avec la même configuration.

Une infrastructure informatique disponible en permanence est indispensable pour le DevOps qui exige des mécanismes de travail agiles et un flux de travaux « workflows » automatisé. Cette infrastructure est nécessaire pour lancer et tester le code développé.

7. Plateforme de gestion de conteneur

De nombreuses entreprises et en particulier les grandes entreprises utilisent les solutions et les plateformes de gestion des conteneurs. En effet, un conteneur est une révolution au niveau du système d'exploitation virtualisé. Au lieu d'un seul espace, il permet de créer des instances dans un espace utilisateur isolé.

Les ressources CPU et RAM utilisées par chaque conteneur sont plus faibles que ceux utilisées par une VM, en plus chaque conteneur fournit un environnement d'exécution sécurisé et isolé pour toute application sans avoir besoin d'un OS et tout en étant portable.

8. Gestion de code source

La gestion du code source est la première phase du processus DevOps. Dans cette étape, s'effectue le contrôle de versions, où les développeurs posent, examinent et gèrent le code.

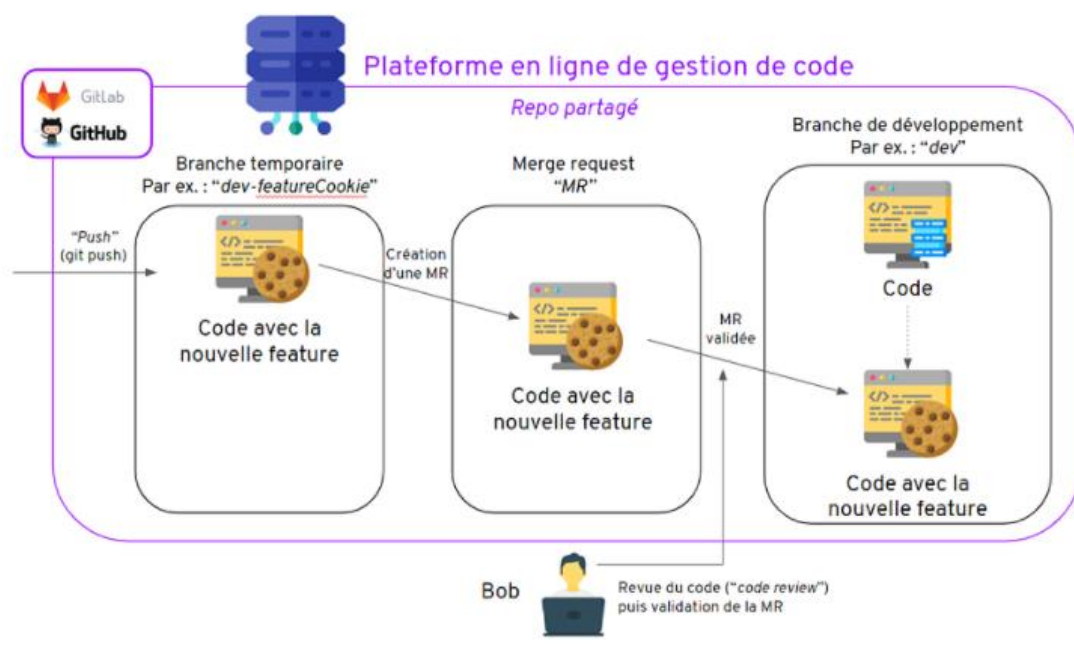


Figure 7 : Principe de gestion de versionning

Gitlab et GitHub sont les deux types d'outil de versionning les plus utilisés, qui servent à créer un historique de ses fichiers. Toutefois, les deux outils sont assez complexes et qui nécessite un certain temps d'adaptation pour bien les comprendre et les manipuler.

9. Automatisation et gestion de configuration

Grâce aux outils d'automatisation et de gestion de configuration, les équipes Ops ont pu éliminer les tâches répétitives avec des configurations automatiques. Ces outils permettent d'automatiser le déploiement des applications. Ainsi, Ansible est apparu pour faciliter l'installation et la modification d'un ou plusieurs serveurs de manière prévisible et déclarative. Cette technologie a pu automatiser les tâches de provisioning dans le cloud et le déploiement d'application.

En effet, l'outil d'automatisation Jenkins, un outil logiciel d'intégration continue, délivre une solution simple d'utilisation pratique dans les configurations. Ce logiciel s'appuie sur ses nombreux plugins et les tests de régression d'automatisation afin de permettre aux équipes de surveiller la réalisation des tâches répétées.

10. Orchestration de containers

Afin de contrôler et d'automatiser le déploiement et le redéploiement des applications, l'outil K8s est considéré comme étant la référence pour l'orchestration de containers, qui est une méthode de virtualisation de système d'exploitation ⁵. Toutefois, il faut mentionner que les containers ne sont pas toujours déployés par les entreprises dans leur plateforme. En effet, les deux outils Kubernetes et Docker Swarm sont toujours comparés vu qu'ils ont les mêmes fonctionnalités de gestion de containers virtuels. Néanmoins, même s'ils présentent des similitudes, ils ont de même de nombreuses différences et particularités. En effet, Docker Swarm s'avère plus avantageux sur quelques points surtout qu'il est simple à manipuler par rapport à Kubernetes. En outre il faut bien souligner que les deux outils sont différents et chacun a ses avantages et ses inconvénients.

II. Etude comparative des technologies adoptées

Pour la mise en place d'un pipeline CI/CD, il sera question dans un premier temps de savoir tous les outils nécessaires tels que, un gestionnaire de code source, orchestrateur de containers, les outils d'automatisation et de gestion de configuration...

Dans cette section du rapport nous comparons les différents outils présents sur le marché et qui répondent à nos nécessités.

1. Outils de gestion de versions

En Cherchant parmi les produits les plus répandus dans le marché des outils de versionning, nous vous proposons un tableau comparatif pour mieux visionner les différentes caractéristiques de chaque outil afin de mieux cerner notre choix d'outils.

Le tableau ci-dessous présente une comparaison entre les deux outils GitHub et Gitlab en se basant sur plusieurs critères :

Caractéristiques	Gitlab	GitHub
Date de création	Septembre 2011	Avril 2008
Plans gratuits	Dépôts publics et privés illimités	Gratuit pour les dépôts publics seulement
Fonctionnalités de révision de code	Disponible	Disponible
Importer et exporter des projets	Oui	Non
Web-hosting	Un outil de cycle de vie Devops basé sur le Web qui	Un service d'hébergement Web pour le contrôle de

	fournir un gestionnaire de référentiel Git	version à l'aide de Git
Popularité	Moins populaire que GitHub	Plus populaire que Gitlab
Plus de fonctionnalité	Offre sa propre intégration continue préconstruite et un support de livraison continue	Fournit diverses intégrations tierces pour une intégration continue et un travail de livraison continue
Suivi du temps	Oui	Non

Tableau 3 : Tableau comparatif Github vs Gitlab

En effet d'après le tableau ci-dessus nous constatons que l'outil Gitlab présente plusieurs avantages, c'est un outil très complet qui offre plusieurs services gratuits et qui peut être installé sur un serveur privé même avec l'édition Community. Toutefois, pour Github, l'installation sur un serveur privé n'est assurée que sur la version entreprise qui est payante ⁶.

2. Outils de gestion de build et d'automatisation

Jenkins, Bamboo et TravisCI sont des outils d'automatisation et de gestion de build les plus populaires sur le marché, chacun d'entre eux présente des avantages et des inconvénients. Le tableau ci-dessous présente une comparaison entre les différents aspects techniques de chaque outil.

Aspect	Jenkins	Bamboo	TravisCI
Plugins	Il prend en charge de nombreux plugins	Il ne supporte pas beaucoup de plugins par rapport à Jenkins	Moins de plugins disponibles

Open source	Oui	Non	Oui
Outil de versionning	Tout	Tout	Github
Simplicité	Non	Oui	Non
Système d'exploitation	Windows, Ubuntu, Red Hat, Mac OS	Windows, Linux, Solaris	Il ne prend pas en charge Windows. Prend uniquement en charge Linux et Mac.

Tableau 4 : Tableau comparatif Jenkins vs Bamboo vs TravisCI

Après avoir exposé une vision globale sur les différences de chaque alternative de produit d'automatisation nous avons une préférence pour Jenkins grâce ses multiples plugins dont nous avons besoin. En plus, il est open source et compatible avec plusieurs systèmes d'exploitation.

3. Outils de monitoring

Pour avoir une plateforme hautement disponible avec des alertes en cas de problème et qui répond aux défis du Devops, nous devons choisir un outil de monitoring qui concorde avec nos besoins.

Le tableau ci-dessous propose une comparaison entre les différents outils de monitoring en se basant sur plusieurs critères :

Aspect	Prometheus	Grafana	Kibana	Nagios
Fonction	Collecte et stockage de métriques,	Outil de visualisation	Outil de visualisation	Outil de visualisation et gestion des

	gestion des alertes			alertes
Ecosystème	Intégration à Grafana	Intégration à Prometheus, InfluxDB et aux services de monitoring des principaux cloud providers	Stock réunissant Elasticsearch, Logstash, Kibana et Beats	centreon-engine
Simplicité	Oui	Oui	Non	Non
Points forts	Couverture fonctionnelle complète, communauté active	Facilité de prise en main, nombreux plugins, Dashboard prêts à l'emploi, version entreprise	L'association avec Elasticsearch, permet de traiter une grosse volumétrie de fichiers de logs	Dashboard prêts à l'emploi, permet de traiter du trafic volumineux
Limites	Durée de rétention réduite	Difficulté dans les premiers temps à configurer les sources de données	Prise en main peu intuitive	Configuration complexe, des modules payantes

Tableau 5 : Tableau comparatif Prometheus vs Grafana vs Kibana vs Nagios

L'étude comparative entre ces différents outils de supervision nous mène à choisir Grafana avec l'intégration de Prometheus vu ses multiples avantages. De plus, les outils de supervision traditionnelle de type Nagios et Centreon, ont montré leurs limites en raison de leurs interfaces dépassées et inactuelles.

4. Outils d'orchestration de containers

Kubernetes et Docker Swarm sont aujourd'hui les principaux outils d'orchestration de conteneurs. Donc, avant de les utiliser dans notre plateforme, nous devons comprendre comment ces deux outils fonctionnent.

Le tableau ci-dessous présente la comparaison entre Kubernetes et Docker Swarm :

	Kubernetes	Docker Swarm
Installation	L'installation est compliquée, Parmi les outils de déploiement est kubeadm	L'installation est très simple
GUI	GUI est le tableau de bord Kubernetes	Il n'y a pas d'interface graphique
API	Peut surmonter les contraintes de Docker et de l'API Docker	Limité aux capacités de l'API Docker
Expérience	Une grande expérience d'exécution de conteneurs Linux chez Google. Le plus souvent déployé en production dans les organisations	Pas de grande expérience dans les déploiements de production à grande échelle.

Evolutivité	Très évolutif	Evolutif
Popularité	Plus populaire que Docker Swarm	Moins populaire que Kubernetes

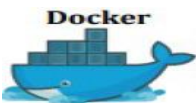
Tableau 6 : Tableau comparatif Kubernetes vs Docker Swarm

Pour l'outil d'orchestration nous avons choisi Kubernetes grâce à ses multiples avantages et fonctionnalités.

Parmi ses principales fonctionnalités nous pouvons citer : l'automatisation de déploiement, la gestion, la mise en réseau, la stabilité et la disponibilité des applications basées sur les conteneurs.

5. Solution de conteneurisation adoptée

Après une recherche approfondie sur les produits les plus réputés dans le marché des conteneurs, nous avons préféré de présenter un tableau comparatif en se basant sur plusieurs critères mentionnés ci-dessous :

	Docker 	Rocket 	LXC/LXD 
Utilisation d'un daemon centralisé	Oui	Non	Oui
Utilisation principale	Applications, commandes simples,	Applications, commandes simples,	OS complets, commandes simples,

	configuration facile	configuration facile	configuration complexes
Simplicité	Oui	Oui	Non
Comptabilité des images	Docker images	Docker images, OCI images	OCI images, full system operating images
Dépôt d'Installation	Dépôt officiel	Dépôt officiel instable ou installation .deb	Dépôt officiel
Sécurité	Téléchargement et vérification automatiques, usage en tant que root	Téléchargement et vérification automatiques	Téléchargement et vérification automatiques (Uniquement pour OS complet)

Tableau 7 : Tableau comparatif Docker vs Rocket vs LXC/LXD

A l'issue de cette comparaison Docker s'avère l'outil qui répond à nos attentes vu qu'il nous propose plusieurs avantages parmi lesquels les conteneurs docker sont légers et faciles à basculer d'environnement à autre presque en quelque seconde.

6. Les outils d'automatisation et gestion de configuration

Dans cette partie, nous présentons un tableau comparatif des outils d'automatisation et de gestion de configuration.

Le tableau ci-dessous offre la comparaison entre Ansible, Puppet et Chef en se basant sur plusieurs critères :

Ansible	Puppet	Chef
Un programmeur n'est pas nécessaire pour gérer cet outil	Un programmeur qui connaît Puppet DSL est nécessaire pour gérer Puppet	Un programmeur Ruby est nécessaire pour gérer l'outil Chef
Ansible signale les erreurs survenues lors de l'installation	Les erreurs ne sont pas signalées dans Puppet lors de l'installation, ce qui rend le processus difficile	Le processus d'installation est difficile dans Chef car les erreurs ne sont pas affichées
Le système de communication est rapide	Un système de communication est lent	Le processus de communication est très lent
Ansible est plus populaire	Puppet n'est pas très populaire	Le chef n'est pas aussi populaire qu'Ansible
Les applications sont facilement déployées à l'aide d'Ansible	Le déploiement d'applications n'est pas si simple	Chef n'effectue pas le déploiement d'application

Tableau 8 : Tableau comparatif Ansible vs Puppet vs Chef

D'après le tableau comparatif n°8, nous avons décidé de choisir Ansible, qui présente plus d'avantage. Cet outil permet la bonne gestion d'une infrastructure et facilite le déploiement des applications avec des scripts qui ne nécessitent pas une bonne connaissance en algorithmique.

Conclusion

Dans le cadre de ce deuxième chapitre nous avons défini et élucidé les notions théoriques en rapport avec notre sujet. Nous avons de même présenté les techniques informatiques usitées

dans le même cadre ainsi que leurs domaines d'utilisation. Ce chapitre présente notre raisonnement afin de mieux choisir les outils qui correspond à nos attentes.

Chapitre 3 : Analyse des besoins et Conception

Introduction

Après avoir entamé les aspects théoriques dans le chapitre précédent, nous focalisons dans ce chapitre sur les besoins fonctionnels et non fonctionnels de notre plateforme de gestion de conteneur. Dans un premier temps, nous présentons la branche fonctionnelle tout en précisant les besoins fonctionnels et non fonctionnels. Dans un deuxième temps nous entamons la deuxième branche conception où nous exposons les étapes préliminaires du développement du système qui devrait répondre à nos objectifs.

I. Analyse des besoins

Dans le cadre de l'analyse des besoins qui est une étape primordiale, nous exposons les besoins fonctionnels et non fonctionnels relatifs à notre projet. Ensuite, nous identifions les acteurs de notre système et nous finissons par une représentation des diagrammes de cas d'utilisation, tout en utilisant UML soutenus d'une description textuelle.

1. Les besoins fonctionnels

Pour le bon fonctionnement de notre plateforme, il est nécessaire de définir concrètement les fonctionnalités qui seront implémentées au sein de notre SI dans le but de les rendre plus appropriées aux besoins de l'entreprise.

Pour ce faire, nous allons supposer que notre plateforme doit être en mesure de fournir les fonctionnalités suivantes :

- ❖ Déploiement rapide de nouveaux environnements à la demande.
- ❖ Gestion automatique des étapes de déploiement de clusters.
- ❖ Haute disponibilité des applications.
- ❖ L'autorisation du système aux administrateurs de gérer les conteneurs à partir des scripts spécifiques
- ❖ Surveillance totale de l'état du notre plateforme.
- ❖ Le stockage des images docker dans un registre est sécurisé
- ❖ Avoir un état complet sur les conteneurs
- ❖ Afficher la liste des projets disponibles sur Gitlab
- ❖ Le même environnement est partagé entre plusieurs développeurs.

- ❖ Le système permet de centraliser les fichiers de configuration des conteneurs dans un outil de versionning.
- ❖ Le développeur est sûr d'avoir la même version des outils que ses collègues en utilisant une image container.
- ❖ Reconstruction rapide et automatique de l'un des composants ou de toute la plateforme en cas du problème.
- ❖ Une plateforme qui gère les micros-services
- ❖ Réalisation de phase de test, réalisées en amont du développement
- ❖ Assurer la qualité de code source avec SonarQube.

2. Les besoins non fonctionnels

Nous définissons dans cette section les critères essentiels pour le bon fonctionnement de notre plateforme. Ainsi, les besoins non-fonctionnels seront traités sur les bases de type de matériel, la performance ou type de conception.

Dans le cadre de notre projet, nous avons essayé de répondre aux besoins non fonctionnels suivants :

- ❖ Sécurité : notre plateforme devra assurer la sécurité des données, des applications et des utilisateurs.
- ❖ La performance : rapidité de la création des environnements.
- ❖ L'évolutivité : nous pouvons améliorer et intégrer des modules pour étendre les fonctionnalités de notre système.
- ❖ Portabilité : rapidité de déplacer et déployer les conteneurs d'un environnement à un autre.
- ❖ Extensibilité : le système doit supporter l'extensibilité de nouveau composant (RAM, stockage) avec l'environnement déjà existant.

3. Identification des acteurs

L'identification des acteurs nous a permis d'identifier trois acteurs dans le système :

— **Développeur** : Il s'agit de l'utilisateur qui va déclencher notre pipeline, il va réagir directement avec Git

— **Testeur** : C'est un acteur qui va bénéficier d'une interface web pour suivre la qualité de l'application dans la phase du test.

— **Administrateur système (Ops)** : L'administrateur système (Ops) est considéré comme l'acteur principal son rôle est la mise en place et la configuration du système de l'infrastructure.

II. Représentation des besoins

Après avoir défini nos besoins et afin de mieux les schématiser et les rendre sensibles, nous présentons sous forme de diagramme de cas d'utilisation les différents éléments.

1. Cas d'utilisation général

Le diagramme de cas d'utilisation met l'accent sur l'importance de l'analyse des besoins métier et permet d'exposer les principales fonctionnalités de façon semi-formelle.

Le diagramme de cas d'utilisation ci-dessous proposera un moyen simple de le faire :

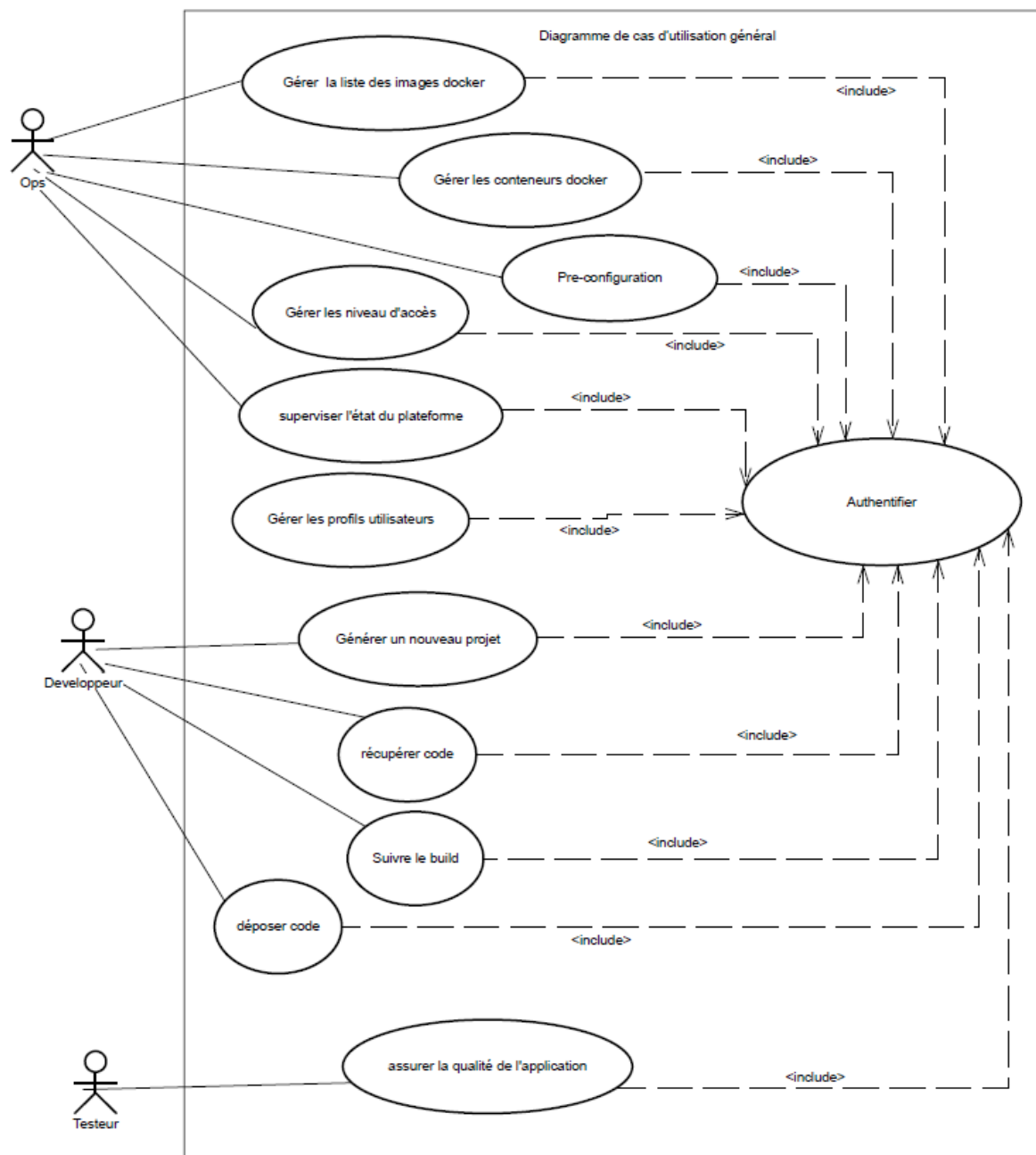


Figure 8 : Diagramme de cas d'utilisation générale

Nous pouvons constater d'après le diagramme de cas d'utilisation générale, les fonctionnalités les plus primordiales du système. Ci-dessous, nous présentons quelques détails sur les fonctionnalités pour chaque acteur.

Le développeur a la capacité d'entreprendre sur le système de plusieurs manières. Il peut :

- ❖ Accéder à l'URL de l'application déployée.

- ❖ Déposer ou récupérer un code.
- ❖ Suivre un build.
- ❖ Générer un nouveau projet qui sera mis en place automatiquement dans un pipeline CI/CD

Pour ce qui est de l'Administrateur système (ops), il a en effet la possibilité d'agir sur le système de plusieurs manières, il peut :

- ❖ Préparer l'environnement pour la mise en place des serveurs
- ❖ Gérer la liste des images docker
- ❖ Configurer notre plateforme pour assurer le bon fonctionnement de notre pipeline
- ❖ Assurer l'état de la plateforme et les mises à jour système
- ❖ Gérer les conteneurs docker

Les testeurs peuvent assurer la qualité de l'application grâce à une interface graphique.

2. Raffinement des cas d'utilisation

Après avoir illustré les différentes fonctionnalités dans notre diagramme de cas d'utilisation globale, nous allons les raffiner dans cette section à travers des diagrammes en illustrant les interactions entre les acteurs.

a. Cas d'utilisation « Gérer la liste des images docker »

La figure ci-dessous présente de façon plus détaillée les différentes fonctions afin de consulter la liste des images docker, c'est ainsi qu'il est possible de créer, supprimer, consulter l'historique d'une image docker.

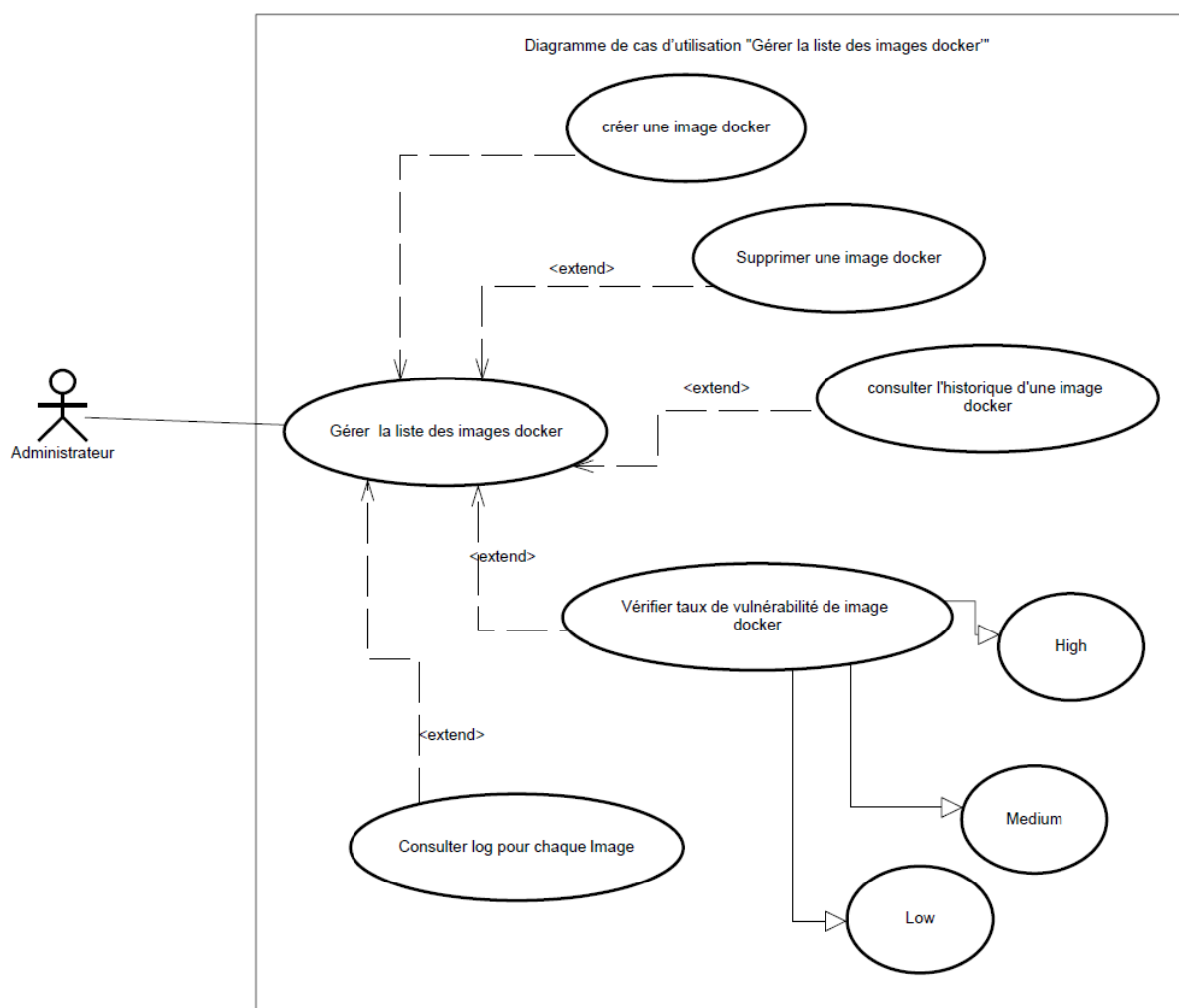


Figure 9 : Diagramme de cas d'utilisation « Gérer la liste des images docker »

Après avoir illustré notre diagramme de cas d'utilisation précédent, nous donnons plus de détails sur les différents cas dans le tableau ci-dessous :

Tableau cas d'utilisation « Gérer la liste des images docker »	
Cas	Gérer la liste des images docker
Description	L'administrateur peut gérer la liste des images docker, qui existe dans le registre : Il peut créer, supprimer, consulter les images docker et avoir des informations sur le taux de vulnérabilité de chaque image en consultant la liste des logs.
Acteur principal	Administrateur

Pré conditions	-Le système est opérationnel -L'administrateur est authentifié
Scénario nominal	-L'administrateur crée une image docker -L'administrateur supprime une image docker -L'administrateur consulte le taux de vulnérabilité d'une image docker -L'administrateur consulte la liste des logs
Scénario alternatif	-L'administrateur n'a pas pu se connecter au registre -Le registre s'est vidé des images Docker si l'administrateur a mal configuré le registre des images. -L'administrateur n'a pas pu supprimer une image si l'administrateur ne supprime pas la signature d'une image.
Post-conditions	-L'administrateur est sur son espace de travail -Le système attend désormais qu'il exécute une nouvelle action -L'administrateur affiche la liste des images docker (scanner et signer)

Tableau 9 : Description Textuelle du cas d'utilisation « Gérer la liste des images docker »

b. Cas d'utilisation « Gestion des profils »

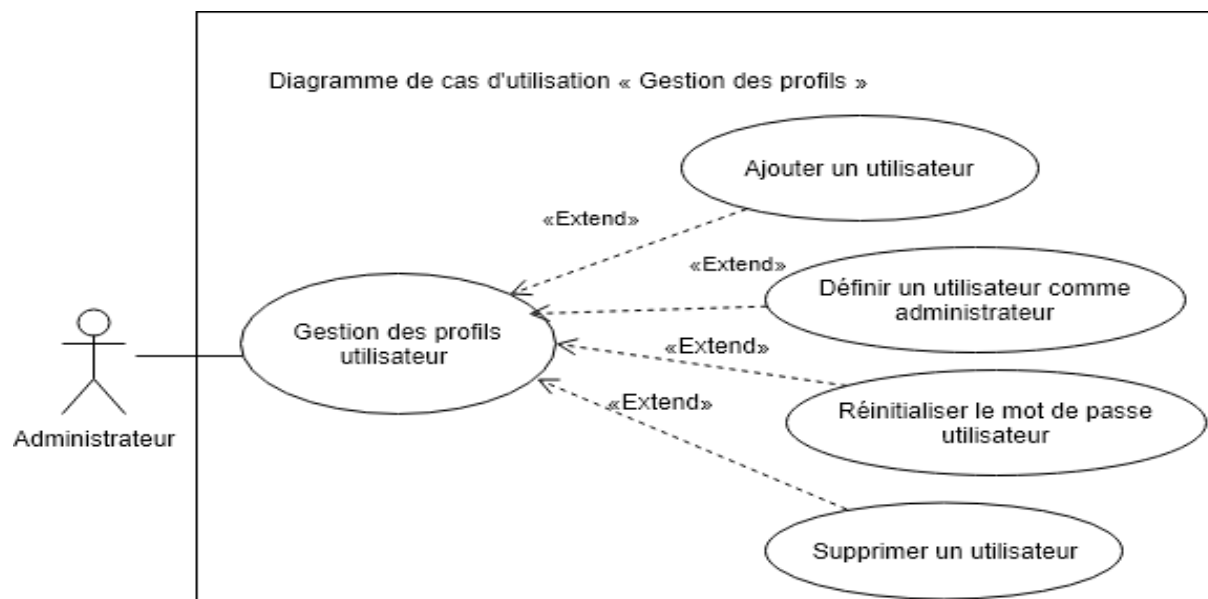


Figure 10 : Diagramme de cas d'utilisation « Gestion des profils »

Dans le tableau ci-dessous nous présentons plus de détails sur les différents cas d'utilisation :

Tableau cas d'utilisation « Gérer les profils utilisateurs »	
Cas	Gérer les profils utilisateurs
Description	L'administrateur peut ajouter un nouvel utilisateur, définir un utilisateur comme : administrateur, membre de groupe (dev, chef de projet, testeur). Il peut aussi supprimer un utilisateur, et réinitialiser le mot de passe ou le verrouiller pour chaque utilisateur
Acteur principal	Administrateur
Pré conditions	-Le système est opérationnel
Scénario nominal	-L'administrateur ajoute un utilisateur -L'administrateur supprime un utilisateur -L'administrateur définit un utilisateur selon leur profil

	-L'administrateur réinitialise le mot de passe utilisateur
Scénario alternatif	-L'administrateur n'a pas pu se connecter au système -Le nom d'utilisateur est déjà utilisé -La longueur maximale de nom et prénom de l'utilisateur doit être de 30 caractères -L'adresse email existe déjà - Le mot de passe doit être dans la norme utilisée - La confirmation de mot de passe ne correspond pas
Post-conditions	-Le système attend désormais qu'il exécute une nouvelle action

Tableau 10 : Description Textuelle du cas d'utilisation « Gérer les profils utilisateurs »

c. Cas d'utilisation « superviser l'état du plateforme »

La figure ci-dessous présente un schéma plus détaillé des différentes fonctions qui supervisent l'état de notre plateforme.

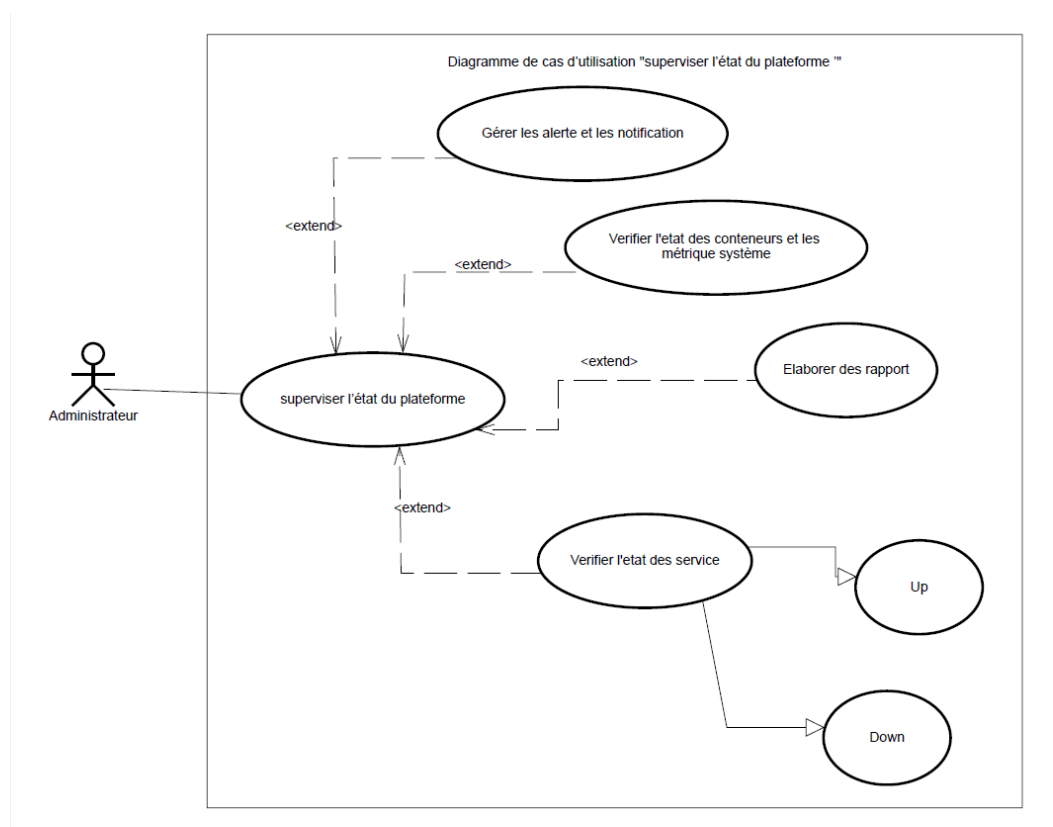


Figure 11: Diagramme de cas d'utilisation « superviser l'état du plateforme »

Dans le tableau ci-dessous nous présentons avec plus de détails le cas d'utilisation « superviser l'état du plateforme » :

Tableau cas d'utilisation « superviser l'état du plateforme »	
Cas	Superviser l'état du plateforme
Description	L'administrateur peut vérifier l'état des services et les gère avec des notifications et des alertes. Il peut aussi élaborer des rapports pour bien évaluer notre plateforme
Acteur principal	Administrateur
Pré conditions	-L'administrateur s'est connecté au système. -Le système est opérationnel
Scénario nominal	-S'authentifier sous Grafana / Prometheus

	-Apporter les modifications souhaitées si besoin à l'application (Gestion des alertes et notification, Dashboard requises). -Consulter les métriques et les logs -Elaborer un rapport
Scénario alternatif	-L'administrateur n'a pas pu se connecter au système -Solution de supervision mal configurée
Post-conditions	-Solution de supervision est créée, configurée et déployée.

Tableau 11 : Description Textuelle du cas d'utilisation « superviser l'état du plateforme »

d. Cas d'utilisation « Générer un nouveau projet »

Au cours de la génération d'un nouveau projet, le développeur peut choisir un Squelette grâce à la plateforme qui va être implémentée automatiquement dans le pipeline de CI/CD.

Pendant la conception d'un nouveau projet, le développeur peut choisir grâce à la plateforme le squelette qui sera (adapter/configurer/déployer) directement dans le « pipeline CI/CD ».

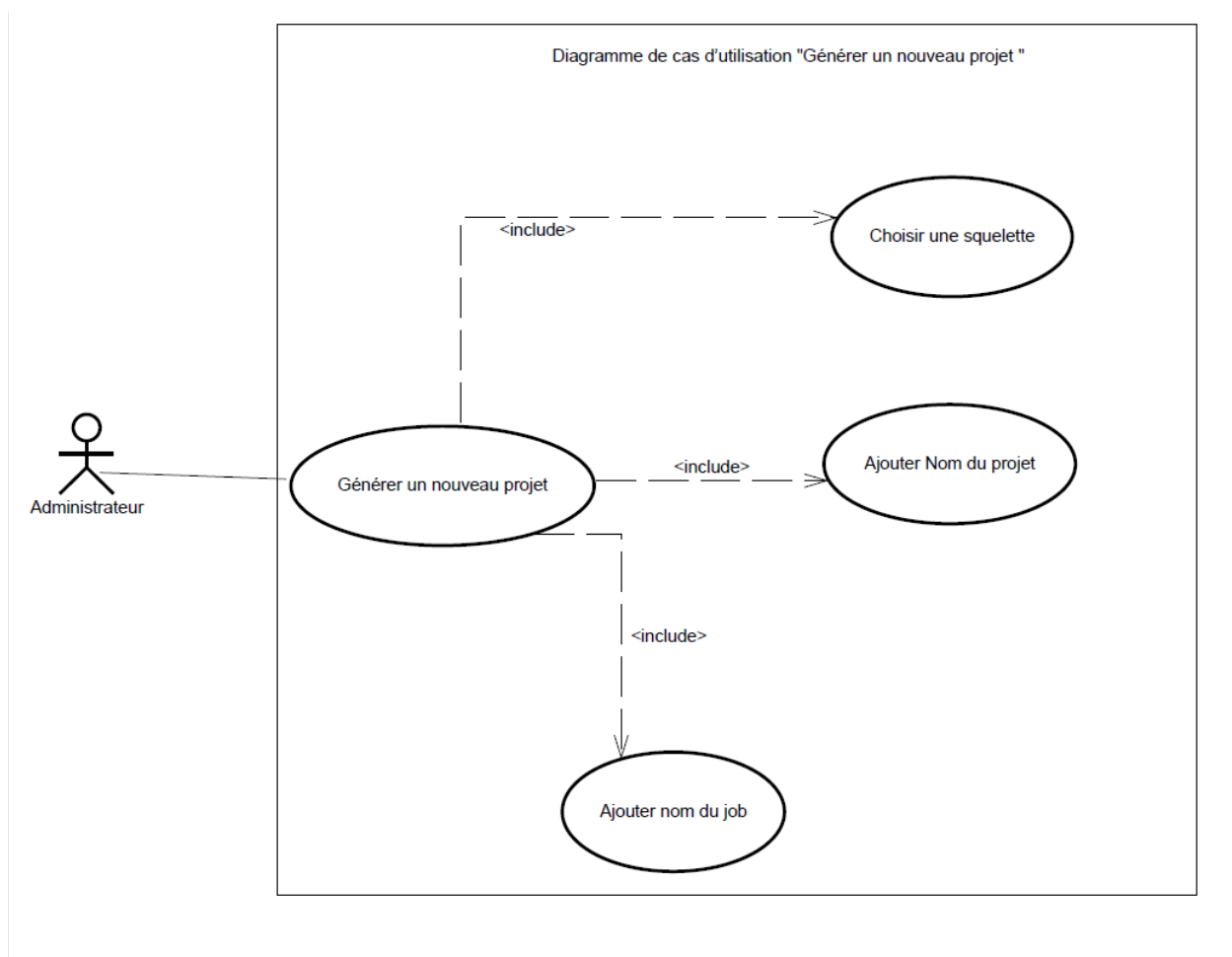


Figure 12 : Diagramme de cas d'utilisation « Générer un nouveau projet »

Tableau cas d'utilisation « Générer un nouveau projet »	
Cas	Générer un nouveau projet
Description	L'administrateur peut vérifier l'état des services et les gérer avec des notifications et des alertes. Il peut aussi élaborer des rapports pour bien évaluer notre plateforme
Acteur principal	Administrateur
Pré conditions	-Avoir un accès aux serveurs Jenkins, Gitlab -Le système est opérationnel -Avoir une connexion SSH ou HTTP établie entre le poste utilisateur et les serveurs Git, Jenkins.

Scénario nominal	- Pour Générer un projet, le développeur clique sur le bouton « générer un projet » et remplit les champs nécessaires. - Après la création de notre projet, Jenkins va déclencher un build sur un conteneur lors d'un changement de code.
Scénario alternatif	-L'administrateur n'a pas pu se connecter au système -Le serveur Git décèle un conflit
Post-conditions	-La connexion entre Jenkins et le serveur Gitlab n'est pas établie. - Le nom choisit par le Développeur existe déjà.

Tableau 12 : Description Textuelle du cas d'utilisation « Générer un nouveau projet »

3. Diagramme séquence système « s'authentifier »

Après avoir illustré les diagrammes de cas d'utilisation et pour mieux interpréter les tableaux ainsi que le scénario principal mené par l'Administrateur système, nous nous référons au diagramme de séquence suivant qui nous montre la première action exécutée par L'administrateur présenté par la figure ci-dessous :

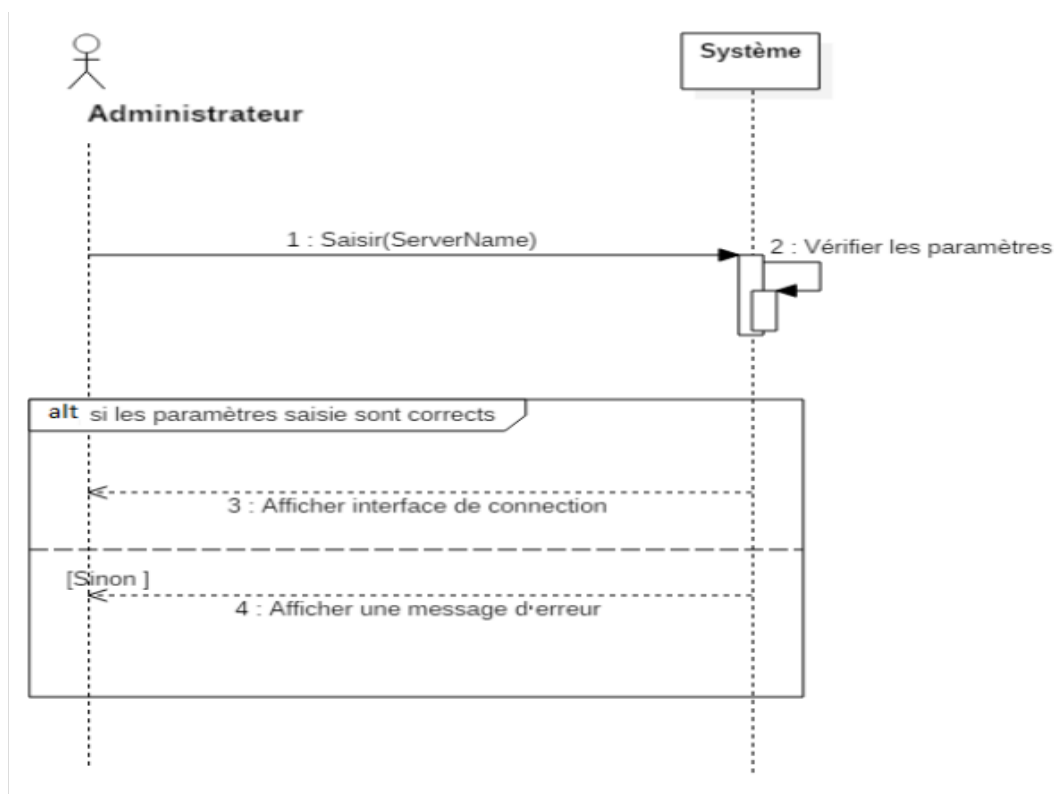


Figure 13 : Diagramme séquence système « s'authentifier »

Pour s'authentifier l'administrateur doit entrer ces informations d'identifications (le login et le mot de passe) correctes, sinon un message d'erreur sera affiché.

4. Diagramme séquence système « Supprimer une image docker »

Le diagramme de séquence dans la figure ci-dessous présente les étapes effectuées par le système pour supprimer une image docker depuis le registre

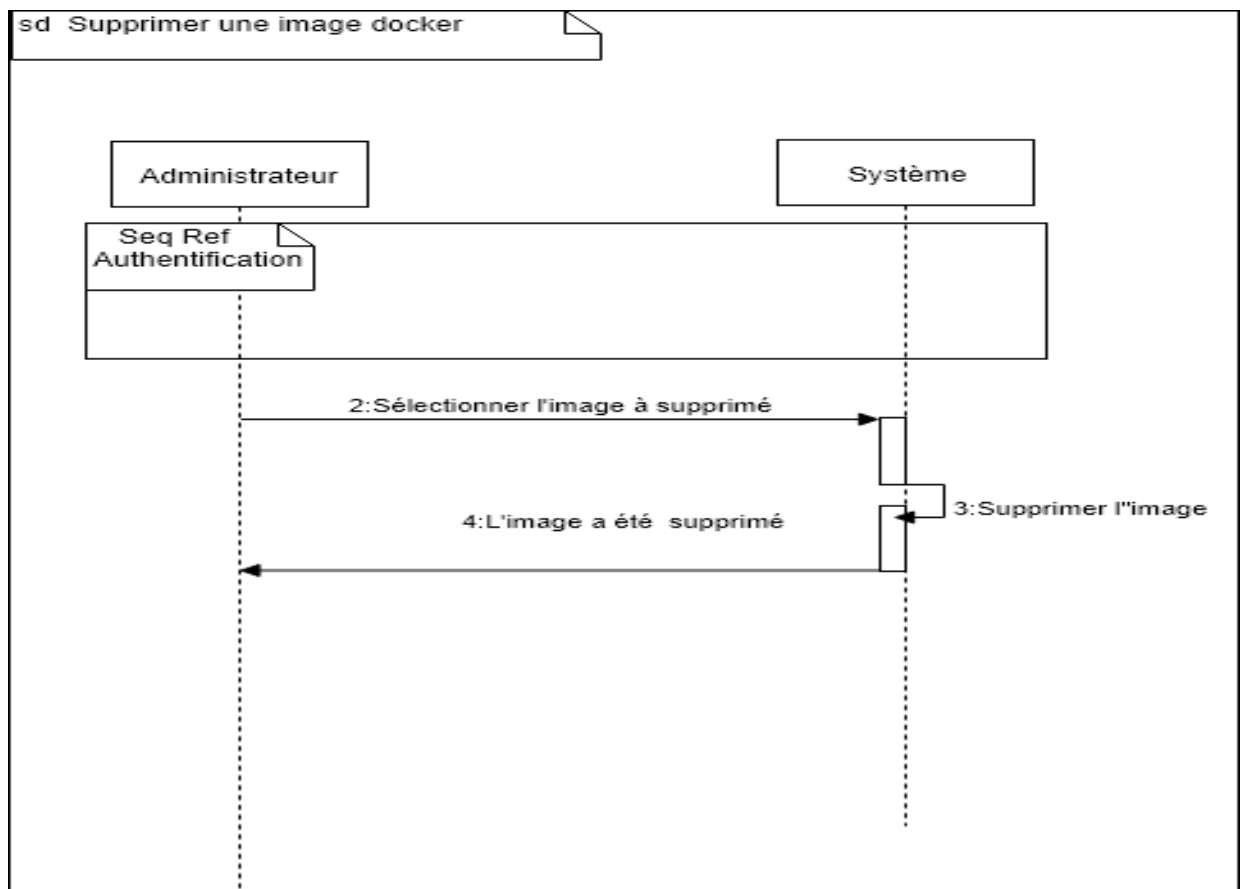


Figure 14 : Diagramme séquence système « Supprimer une image docker »

L'administrateur doit s'authentifier tout d'abord pour qu'il puisse accéder au registre des images docker et le supprimer.

III. Conception

Après avoir défini les différents besoins de projet, il est nécessaire de présenter la conception dans laquelle nous illustrons l'architecture globale du projet. Nous commençons par exposer l'architecture physique et le diagramme de déploiement, ensuite, nous présentons la description de la vue dynamique de notre plateforme avec les diagrammes d'activités de chaque volet.

1. Architecture générale de la solution

Pour répondre aux besoins définis auparavant, voici une architecture technique de notre pipeline CI/CD.

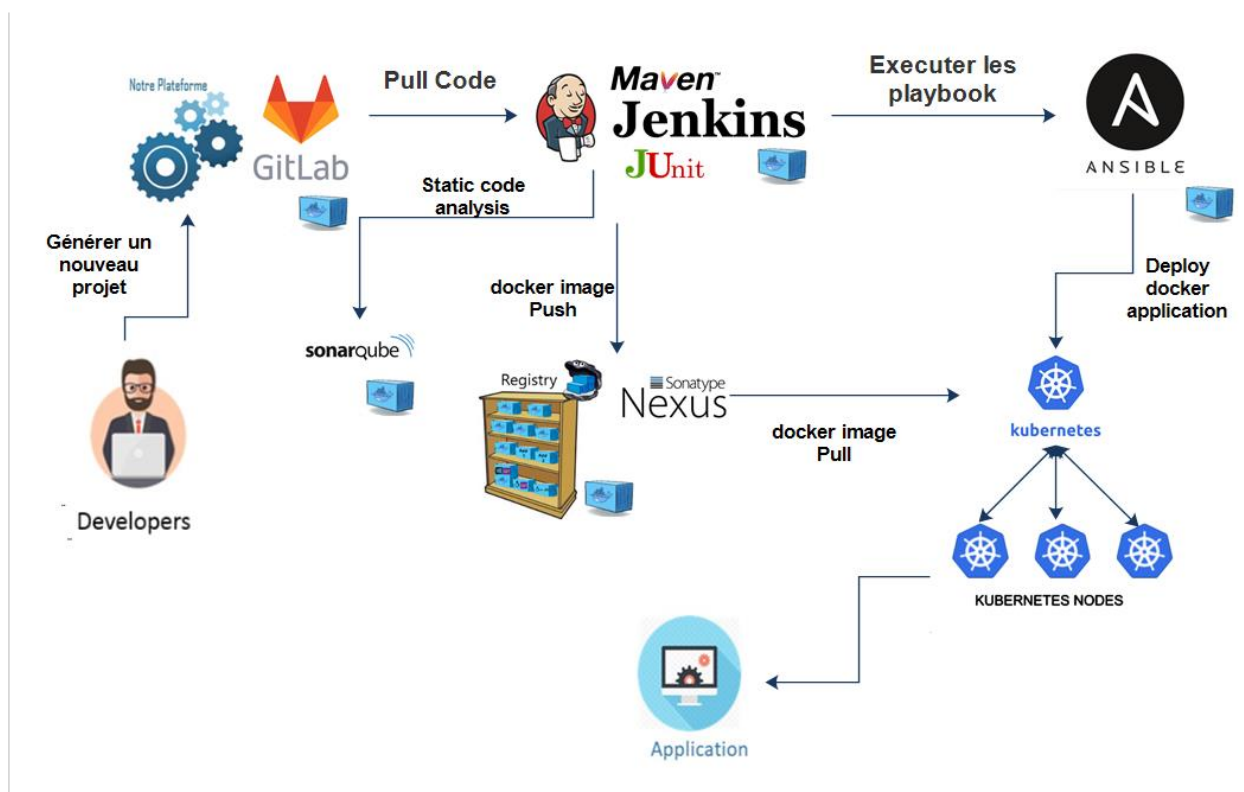


Figure 15 : Architecture générale de la solution

L'architecture représente les relations des différents composants de notre pipeline, la plupart des composants sont conteneurisés dans des conteneurs docker qui peuvent être, montés et supervisés par un orchestrateur K8s. Cette architecture permet de gagner plusieurs avantages notamment par rapport au gain des ressources.

Dans le but d'automatiser la création de notre projet et l'implémenter dans une chaîne CI/CD basée sur les serveurs mentionnés précédemment, Nous présentons les phases du traitement de chaque composant dans notre architecture :

- 1 Push des modifications de codes
- 2 Si la mise à jour de code détecté au niveau de Gitlab, le Job de build Jenkins est démarrée, le build est exécuté avec l'outil Maven
- 3 Jenkins lance les tests unitaires à la suite de la compilation avec l'outil JUnit
- 4 Mesurer la qualité du code avec l'outils SonarQube
- 5 Jenkins upload image Docker vers Nexus
- 6 Ansible exécute les Playbook
- 7 Ansible procède au déploiement dans Kubernetes

- 8 Kubernetes pull les dernières releases à partir de Nexus et assure l'orchestration de chaque Nœud.

Comme nous l'avons déjà mentionné précédemment dans notre rapport notre plateforme est supervisée par les outils Grafana et Prometheus, Ci-dessous nous présentons le schéma explicatif de la partie monitoring de notre système.

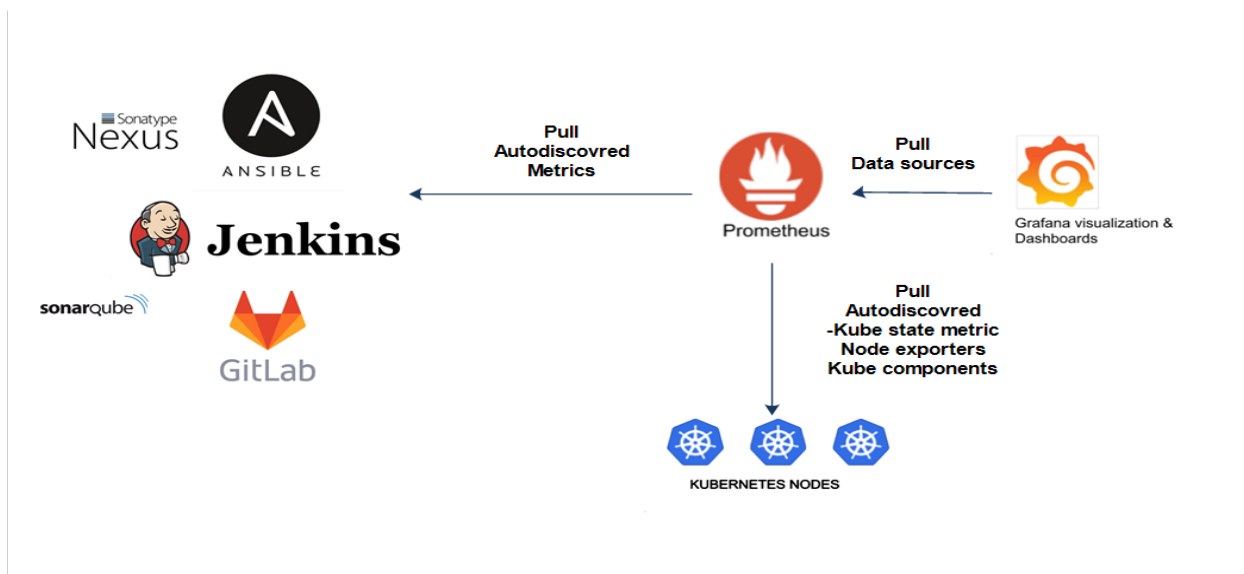


Figure 16 : Architecture générale de la solution

2. Diagramme de déploiement

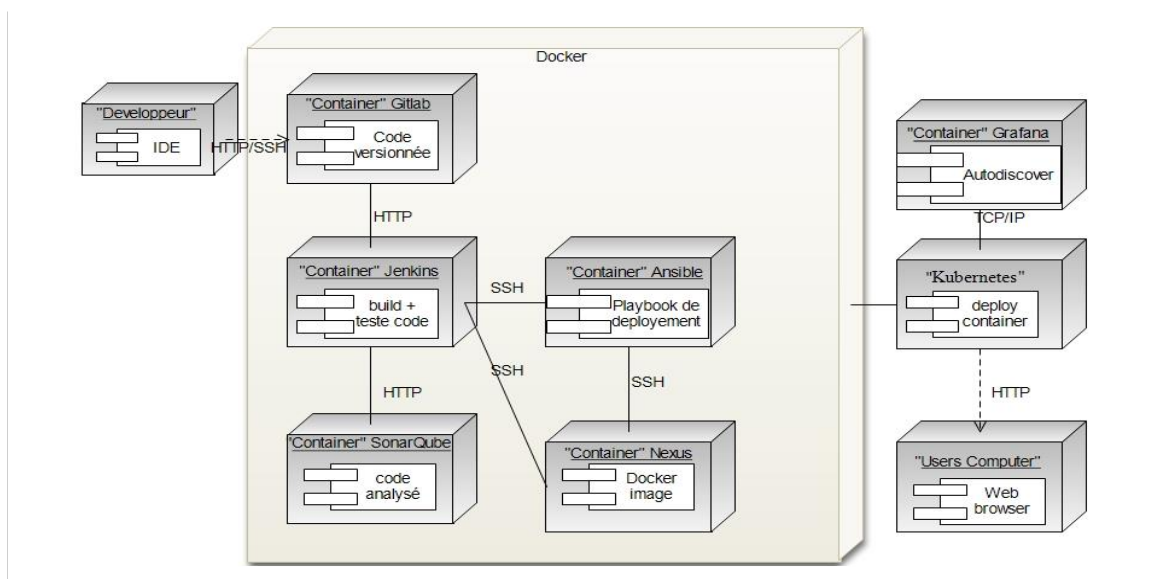


Figure 17 : Diagramme de déploiement

La figure 17 représente le schéma de notre architecture physique. Ainsi nous pouvons bien voir que nos différents outils (Jenkins, Gitlab, SonarQube, Nexus et Ansible) sont bien conteneurisés et orchestrés par le logiciel Kubernetes et la communication entre les différents composants est assurée par le protocole de communication (http et ssh).

3. Aspect dynamique de la solution

Afin de mieux gérer la corrélation entre les différents éléments de notre nouvelle infrastructure, nous présentons une vue dynamique de notre système.

a. Diagramme d'activité « Authentification »

Pour réussir l'accès à notre système, l'administrateur doit s'authentifier pour qu'il puisse gérer les différentes fonctionnalités, si l'opération échoue l'accès à l'application est interdit à l'administrateur

Parmi les fonctionnalités gérables par l'administrateur, on cite :

- ❖ Gérer la liste des images docker
- ❖ Gérer les profils utilisateurs
- ❖ Superviser l'état de la plateforme

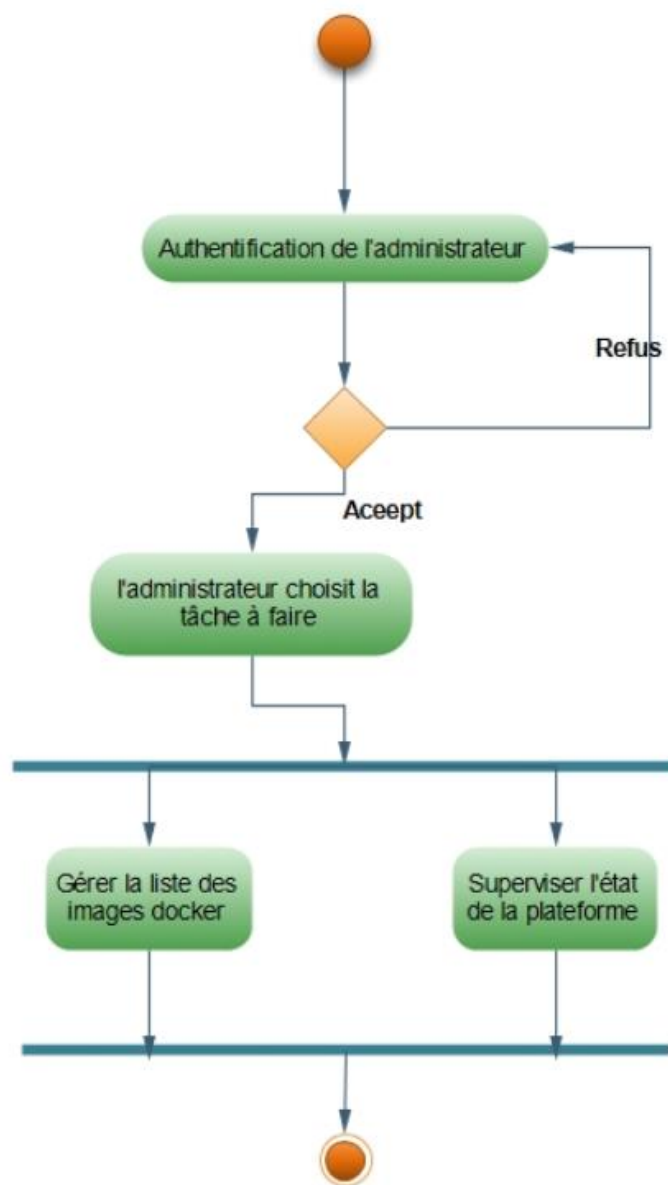


Figure 18 : Diagramme d'activité « Authentification »

b. Diagramme d'activité « Gérer la liste des images docker »

Afin de mieux comprendre l'activité 'Gérer la liste des images docker', nous exposons dans la figure ci-dessous un diagramme d'activité qui va définir les différentes étapes à faire au niveau de chaque fonctionnalité.

Parmi les fonctionnalités gérables par l'administrateur, on cite :

- ❖ Suppression des images docker

- ❖ Consulter la liste des logs des images docker
- ❖ Créer une image docker
- ❖ Consulter l'historique des images docker

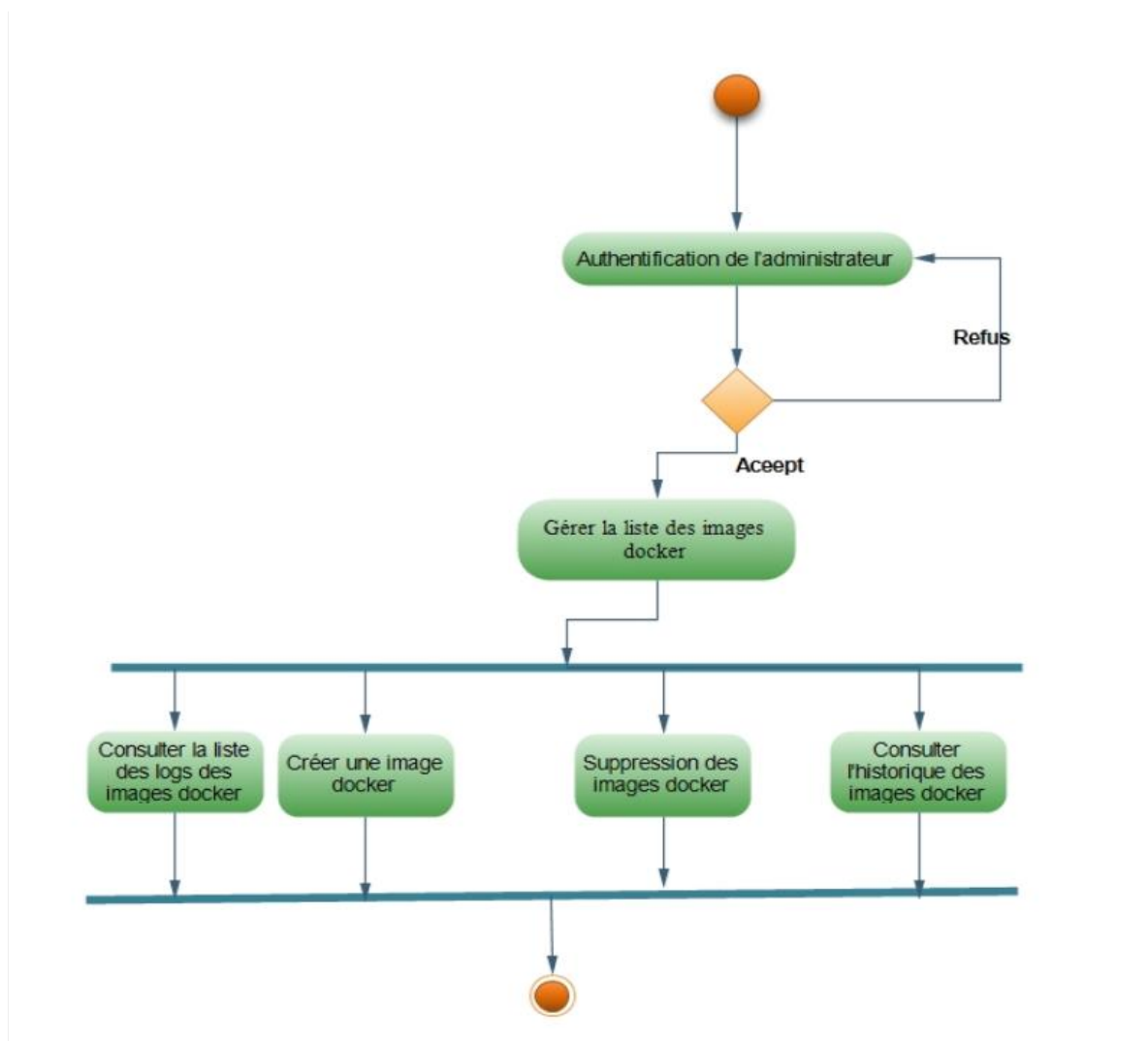


Figure 19: Diagramme d'activité « Gérer la liste des images docker »

Conclusion

Dans ce chapitre, nous avons défini les différents besoins fonctionnels et non fonctionnels de notre plateforme et présenté les différentes architectures techniques et physique de notre solution que nous les avons illustrées sous forme de digrammes d'activités afin de mieux schématiser l'aspect dynamique de cette dernière.

Le chapitre suivant nous entamerons la partie réalisation qui est la mise au point de notre travail.

Chapitre 4 : Réalisation

Introduction

Dans la partie précédente nous avons entamé les différentes questions liées à la conception du projet et nous avons limité les objectifs et les besoins à satisfaire.

Dans ce chapitre nous exposons en premier lieu l'environnement de travail, notamment l'environnement logiciel et matériel de notre projet, ensuite nous présentons les différentes technologies utilisées dans la mise en place de l'infrastructure. Finalement, nous exposons une démonstration de notre travail ainsi que des documents graphiques (imprime-écran) des différentes interfaces.

I. Environnement de travail

Dans cette partie, nous présentons les principaux outils matériel et logiciel utilisés dans le cadre de ce projet.

1. Environnement matériel

Pour ce qui est de l'environnement matériel, nous avons à notre disposition un ordinateur DELL dont les caractéristiques techniques sont :

- Mémoire : 32 GB RAM
- Processeur : Intel(R) Core(TM) i7-9750 CPU @ 2,60GHz 2.59 GHZ
- Disque dur : 256 GB SSD + 1 To

2. Environnement logiciel

Pour ce qui est de l'environnement logiciel utilisé pour la réalisation du projet nous pouvons citer :

- VMware Workstation Pro 15
- Système d'exploitation : Red Hat Enterprise Linux « Centos7 »
- Navigateur Web : Mozilla Firefox
- Outil de modélisation UML : PowerAMC
- Outil de modélisation des figures : Edraw MAX

3. Choix Technologiques

Nous illustrons dans le tableau ci-dessous les technologies utilisées pour la mise au point de notre plateforme tout en respectant les besoins déjà cités.

Technologie	Fonctionnalités
	Docker est une technologie qui implémente une application dans un ou plusieurs containers
	Jenkins est un serveur qui permet l'automatisation des tâches d'intégration, de livraison et de déploiement
	Gitlab est un outil de gestion de version pour la gestion de version de code
	Nexus est un outil de gestion des dépôts des images docker
	Maven est un outil de construction de projets (build).
	SonarQube permet non seulement de mesurer la qualité de votre projet de plusieurs façons, mais également de mettre en évidence les problèmes nouvellement introduits.
	Ansible est une plate-forme logicielle libre pour la configuration et la gestion des ordinateurs
	Prometheus est un logiciel libre de surveillance informatique et générateur d'alertes
 Grafana visualization & Dashboards	Grafana permet la visualisation de données. Il permet de réaliser des tableaux de bord et des graphiques depuis plusieurs sources dont des bases de données temporelles.
 kubernetes	Kubernetes est un système d'orchestration pour automatiser le déploiement, la mise à l'échelle et la gestion d'applications informatiques

	Spring est un Framework open source pour construire et définir l'infrastructure d'une application
---	--

Tableau 13 : Technologies utilisés

II. Principaux développements

Dans cette partie nous exposons les différentes actions d'exécution, à partir des imprimés d'écrans qui présentent la description des fonctionnalités du système.

1. Génération d'un nouveau Pipeline

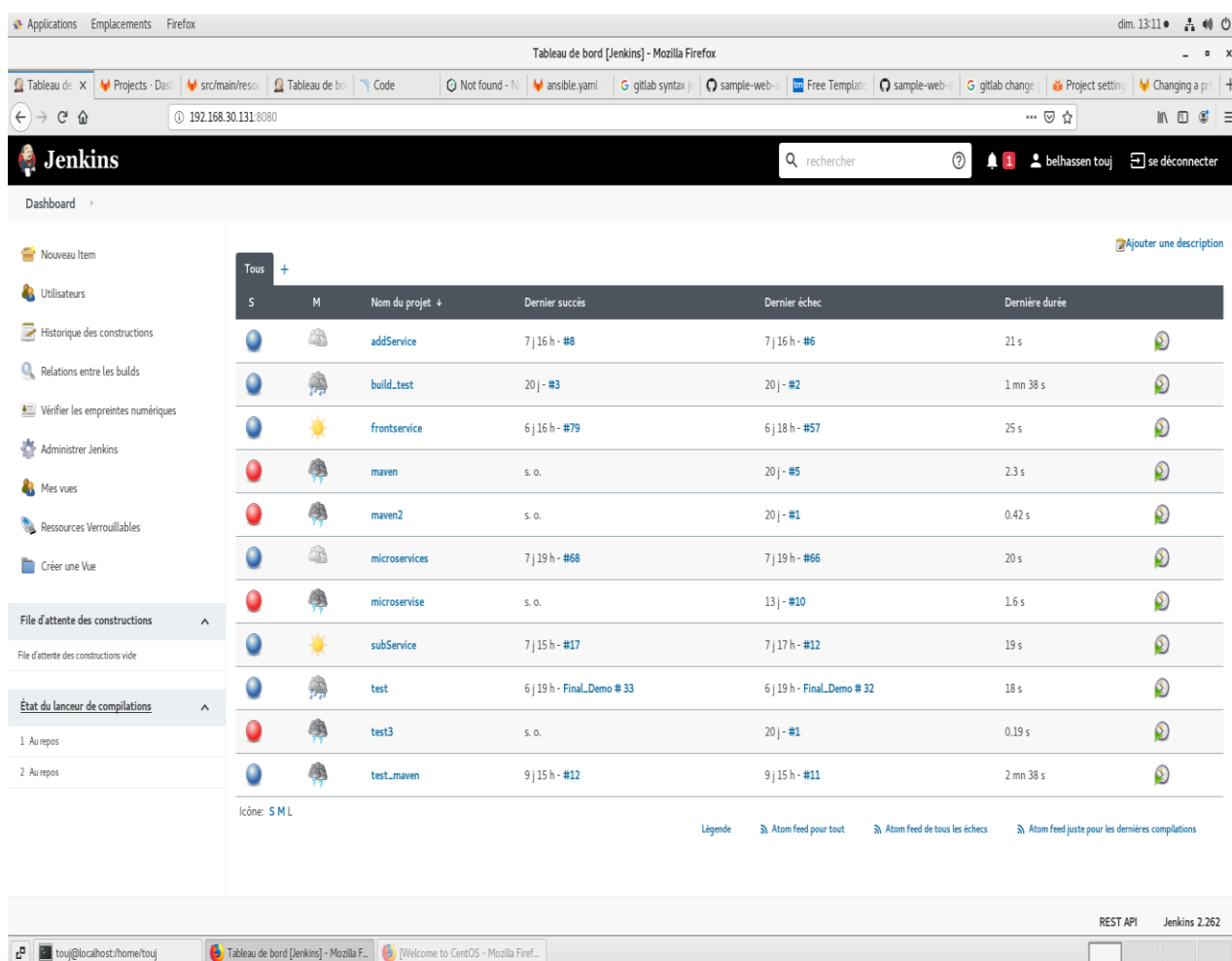


Figure 20 : capture écran montrant les jobs dans Jenkins

La figure 20 présente l'interface du serveur Jenkins en manifestant les jobs exécutés. Parmi ses projets nous présentons les 3 « Job » de notre projet, le premier « addService » qui a pour but de faire l'addition, le deuxième « subService » qui a pour but de faire soustraction et le dernier job « frontservice » qui a pour but d'afficher le résultat de job addService et subService.

La figure ci-dessous expose l'interface du serveur Jenkins qui permet de créer un pipeline, tout d'abord, nous choisissons le nom du job après nous configurons notre script.

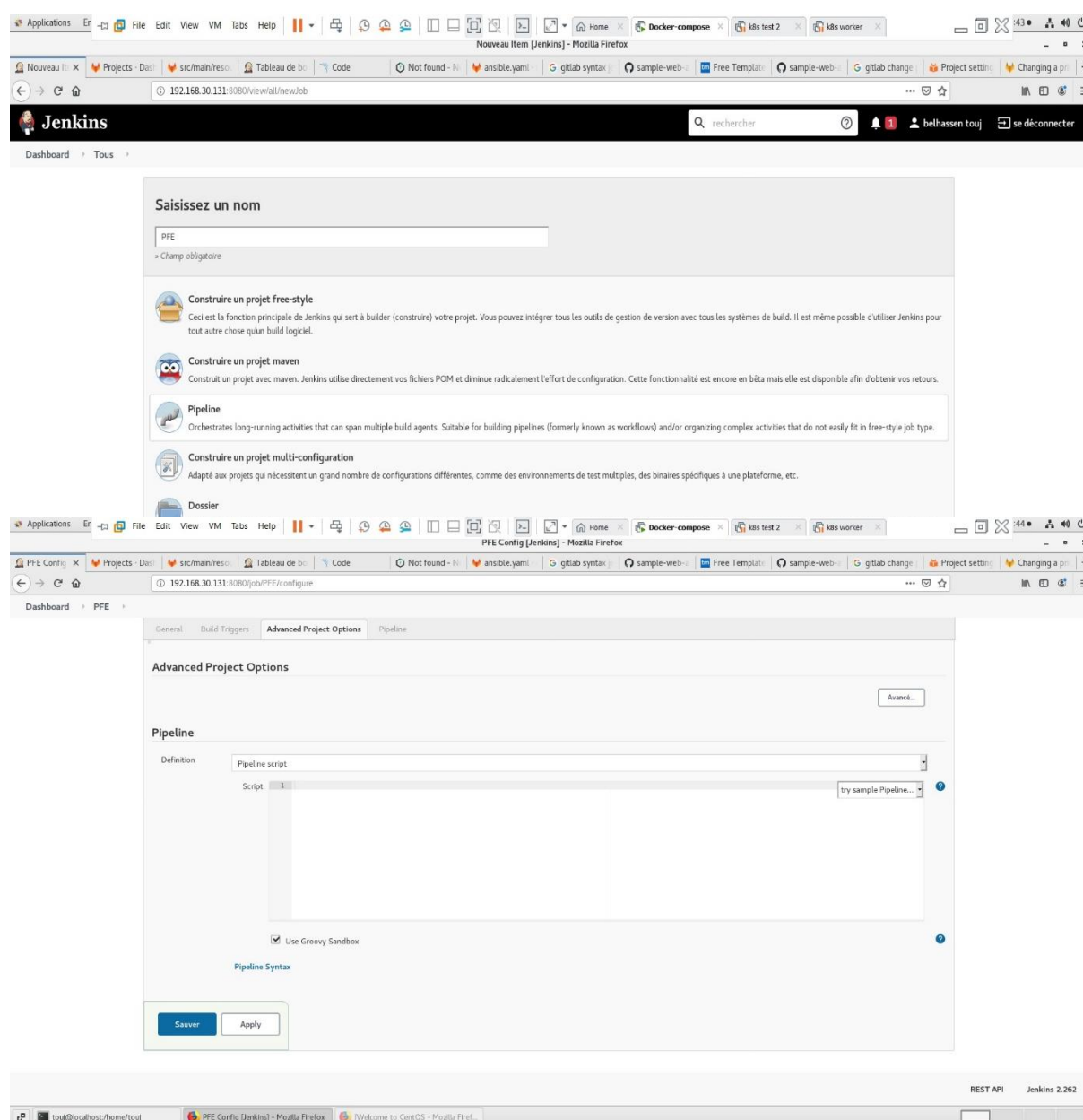


Figure 21 : Les étapes à faire pour générer un Job

Le rapport du build et le Stage View du pipeline de Job « frontservice » est représenté par la figure ci-dessous, pour savoir le statut de notre build.

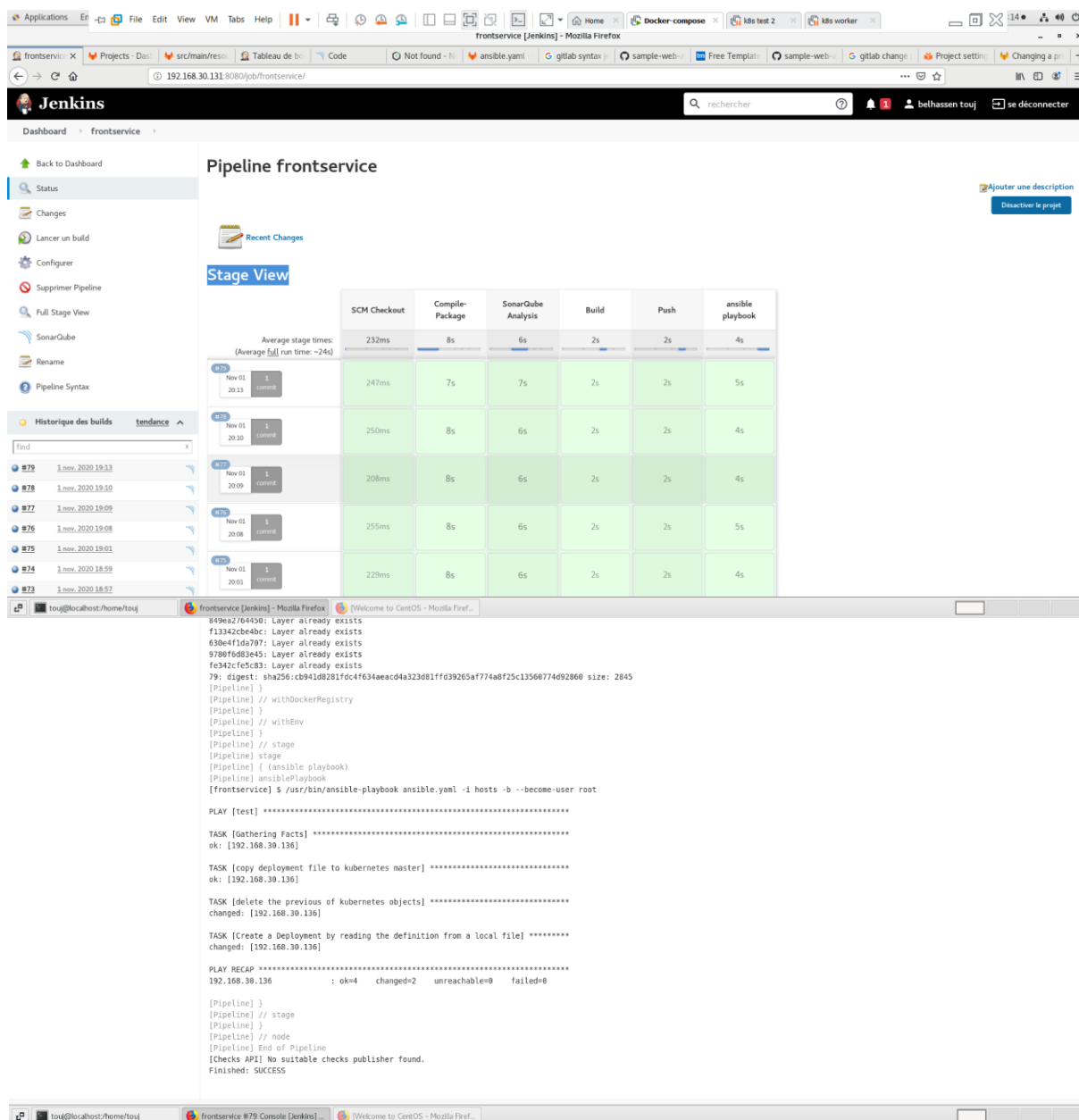


Figure 22 : Rapport du build et le Stage View du pipeline

2. Intégration et Déploiement Continu

Pour la mise en place d'un nouveau projet, il est important de l'implémenter dans un pipeline d'intégration et déploiement continu.

a. Intégration Continue

Afin d'avoir un pipeline bien configuré, nous reposons sur la mise en place d'un serveur de gestion de version « Gitlab » qui va déclencher notre pipeline à chaque modification du code source (push).

Nous constatons d'après la figure ci-dessous que notre Projet est mis en place dans notre serveur Gitlab.

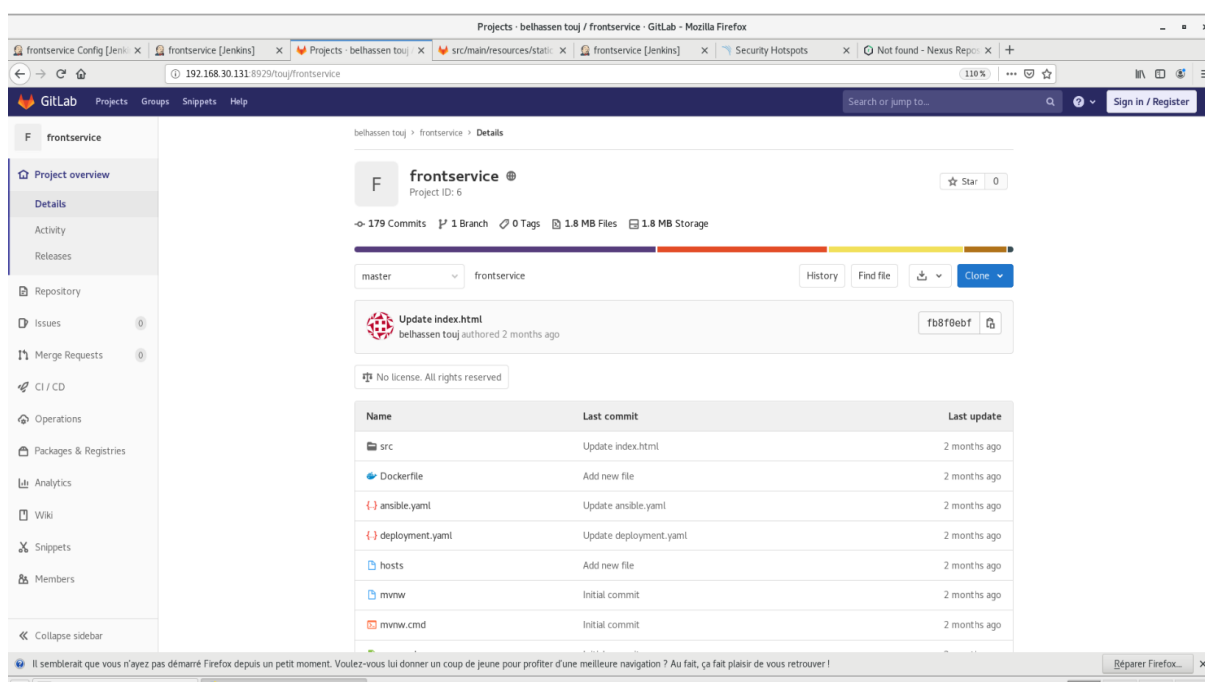


Figure 23 : Le projet implémenté sur Gitlab

Le changement dans le code source est représenté dans la figure ci-dessous, qui vise à faire un « push » manuellement pour déclencher notre pipeline.

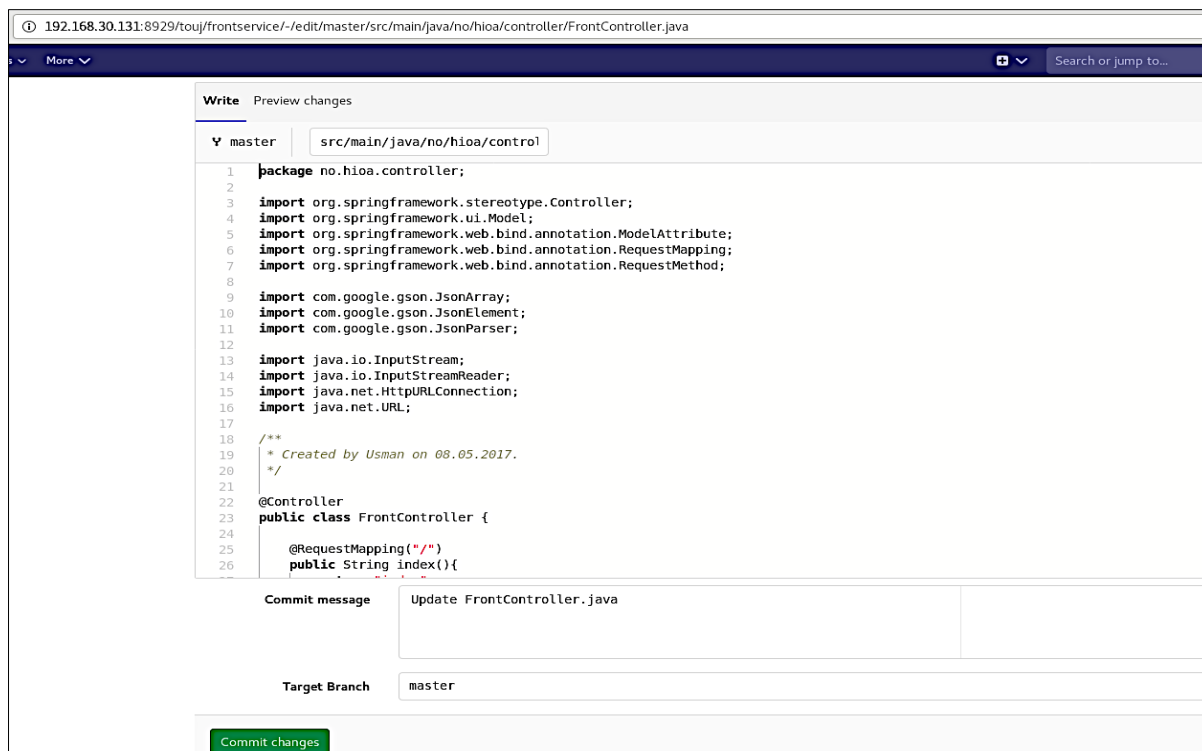


Figure 24 : Changement du code source manuellement

Nous constatons d'après la figure ci-dessus que notre Pipeline est déclenché automatiquement.

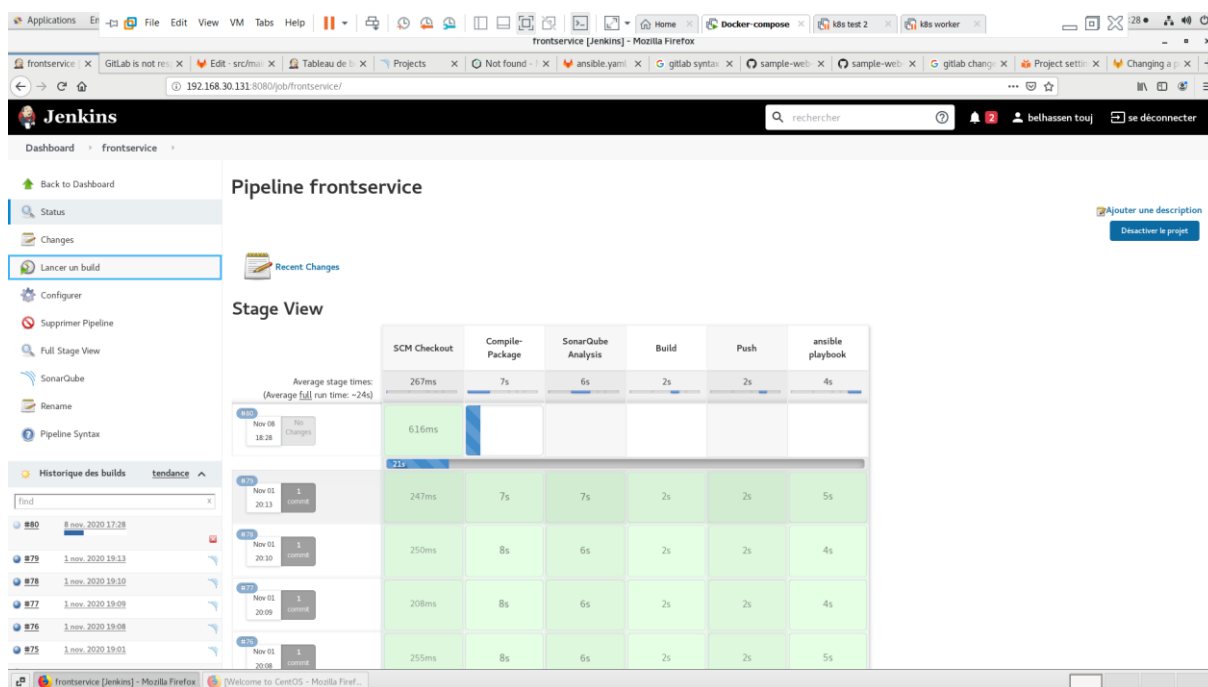


Figure 25 : Déclenchement d'un build automatiquement

Pendant que le Pipeline fonctionne, le Build est exécuté avec l'outil de construction Maven ce qui va déclencher en même temps un test du code source grâce au Framework JUnit.

```
-----
T E S T S
-----
13:47:55.170 [main] DEBUG org.springframework.test.context.junit4.SpringJUnit4ClassRunner - SpringJUnit4ClassRunner constructor called with [class
no.hioa.FrontServiceApplicationTests]
13:47:55.178 [main] DEBUG org.springframework.test.context.BootstrapUtils - Instantiating CacheAwareContextLoaderDelegate from class
[org.springframework.test.context.cache.DefaultCacheAwareContextLoaderDelegate]
13:47:55.192 [main] DEBUG org.springframework.test.context.BootstrapUtils - Instantiating BootstrapContext using constructor [public
org.springframework.test.context.support.DefaultBootstrapContext(java.lang.Class,org.springframework.test.context.CacheAwareContextLoaderDelegate)]
13:47:55.214 [main] DEBUG org.springframework.test.context.BootstrapUtils - Instantiating TestContextBootstrapper for test class
[no.hioa.FrontServiceApplicationTests] from class [org.springframework.boot.test.context.SpringBootTestContextBootstrapper]
13:47:55.233 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper - Neither @ContextConfiguration nor @ContextHierarchy found
for test class [no.hioa.FrontServiceApplicationTests], using SpringBootContextLoader
13:47:55.238 [main] DEBUG org.springframework.test.context.support.AbstractContextLoader - Did not detect default resource location for test class
[no.hioa.FrontServiceApplicationTests]: class path resource [no/hioa/FrontServiceApplicationTests-context.xml] does not exist
13:47:55.239 [main] DEBUG org.springframework.test.context.support.AbstractContextLoader - Did not detect default resource location for test class
[no.hioa.FrontServiceApplicationTests]: class path resource [no/hioa/FrontServiceApplicationTestsContext.groovy] does not exist
13:47:55.239 [main] INFO org.springframework.test.context.support.AbstractContextLoader - Could not detect default resource locations for test class
[no.hioa.FrontServiceApplicationTests]: no resource found for suffixes {-context.xml, Context.groovy}.
13:47:55.240 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils - Could not detect default configuration classes for
test class [no.hioa.FrontServiceApplicationTests]: FrontServiceApplicationTests does not declare any static, non-private, non-final, nested classes
annotated with @Configuration.
13:47:55.290 [main] DEBUG org.springframework.test.context.support.ActiveProfilesUtils - Could not find an 'annotation declaring class' for annotation type
[org.springframework.test.context.ActiveProfiles] and class [no.hioa.FrontServiceApplicationTests]
13:47:55.299 [main] DEBUG org.springframework.core.env.StandardEnvironment - Adding [systemProperties] PropertySource with lowest search precedence
```

Figure 26 : Déclenchement d'un test avec JUnit

Cette figure montre que le build et le test sont exécutés avec succès.

Tableau de bord > frontservice > #115

```
[[1;34mINFO[m] Sensor javaSquidSensor [java] (done) | time=118ms
[[1;34mINFO[m] Sensor SurefireSensor [java]
[[1;34mINFO[m] parsing [/var/jenkins_home/workspace/frontservice/target/surefire-reports]
[[1;34mINFO[m] Sensor SurefireSensor [java] (done) | time=60ms
[[1;34mINFO[m] Sensor JavaXmlSensor [java]
[[1;34mINFO[m] 1 source files to be analyzed
[[1;34mINFO[m] Sensor JavaXmlSensor [java] (done) | time=121ms
[[1;34mINFO[m] ----- Run sensors on project
[[1;34mINFO[m] 1/1 source files have been analyzed
[[1;34mINFO[m] Sensor Zero Coverage Sensor
[[1;34mINFO[m] Sensor Zero Coverage Sensor (done) | time=6ms
[[1;34mINFO[m] Sensor Java CPD Block Indexer
[[1;34mINFO[m] Sensor Java CPD Block Indexer (done) | time=14ms
[[1;34mINFO[m] SCM Publisher No SCM system was detected. You can use the 'sonar.scm.provider' property to explicitly specify it.
[[1;34mINFO[m] CPD Executor 2 files had no CPD blocks
[[1;34mINFO[m] CPD Executor Calculating CPD for 1 file
[[1;34mINFO[m] CPD Executor CPD calculation finished (done) | time=8ms
[[1;34mINFO[m] Analysis report generated in 35ms, dir size=34 KB
[[1;34mINFO[m] Analysis report compressed in 14ms, zip size=11 KB
[[1;34mINFO[m] Analysis report uploaded in 47ms
[[1;34mINFO[m] ANALYSIS SUCCESSFUL, you can browse http://192.168.30.131:9000/dashboard?id=no.hioa%3Afrontservice
[[1;34mINFO[m] Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
[[1;34mINFO[m] More about the report processing at http://192.168.30.131:9000/api/ce/task?id=AXcv9IL9wY8CsXmqCRpm
[[1;34mINFO[m] Analysis total time: 2.542 s
[[1;34mINFO[m] [1m-----[m
[[1;34mINFO[m] [1;32mBUILD SUCCESS[m
[[1;34mINFO[m] [1m-----[m
[[1;34mINFO[m] Total time: 4.571 s
[[1;34mINFO[m] Finished at: 2021-01-23T13:48:07Z
[[1;34mINFO[m] [1m-----[m
[Pipeline] }
```

Figure 27 : Déclenchement d'un test Qualité avec SonarQube

Suite à l'étape précédente un test de qualité du code de notre projet va se déclencher et le résultat sera afficher dans une Dashboard de SonarQube (Figure 27)

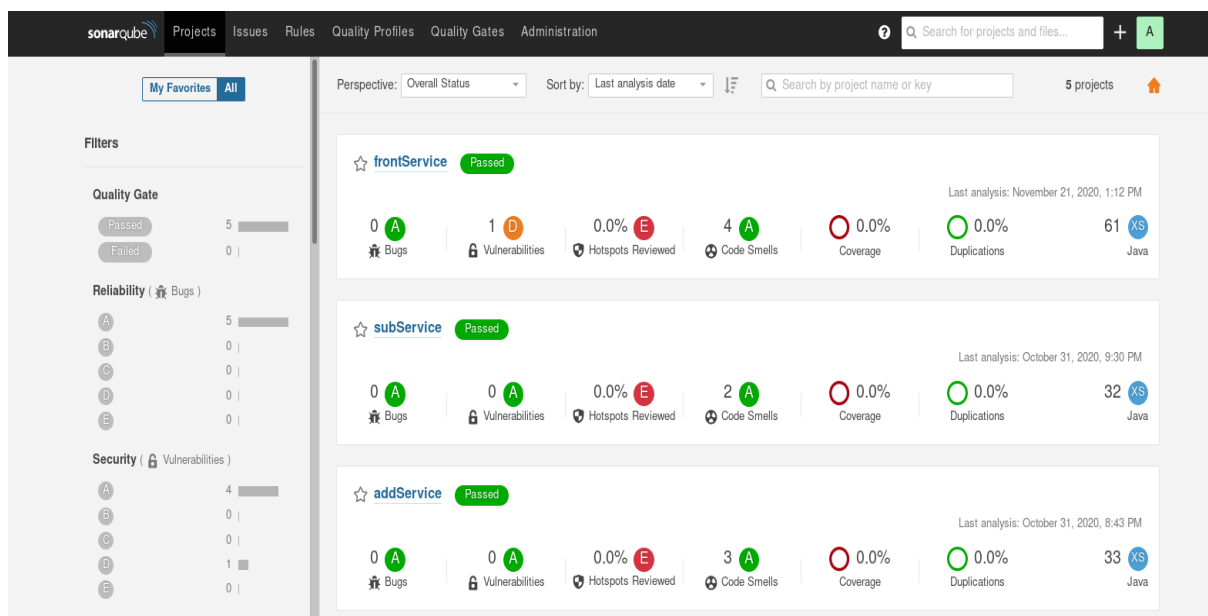


Figure 28 : SonarQube Dashboard

D'après la figure 28 nous voyons le Dashboard de SonarQube qui présente le rapport de la qualité du code du projet.

b. Déploiement Continu

Suite à l'intégration continue, nous avons l'étape du déploiement continu qui consiste à déployer l'application.

```
[Pipeline] isUnix
[Pipeline] sh
+ docker build -t frontservice:79 .
Sending build context to Docker daemon 77.63MB

Step 1/5 : FROM tomcat
----> 891fcd9c5b3a
Step 2/5 : WORKDIR webapps
----> Using cache
----> 0d155d0edd5d
Step 3/5 : COPY target/frontservice-0.0.1-SNAPSHOT.war .
----> 6df01e524e1c
Step 4/5 : RUN rm -rf ROOT && mv frontservice-0.0.1-SNAPSHOT.war ROOT.war
----> Running in 7221aa871ad2
Removing intermediate container 7221aa871ad2
----> 715e1ef0d238
Step 5/5 : ENTRYPOINT ["sh", "/usr/local/tomcat/bin/star
----> Running in 75c423cfc150
Removing intermediate container 75c423cfc150
----> 4f80fc0610d0
Successfully built 4f80fc0610d0
Successfully tagged frontservice:79
```

Figure 29 : Déclenchement d'un Build d'image docker

La première étape, est de créer une image Docker avec l'artéfact généré avec le Build exécuté pendant l'étape précédente (Figure 29).

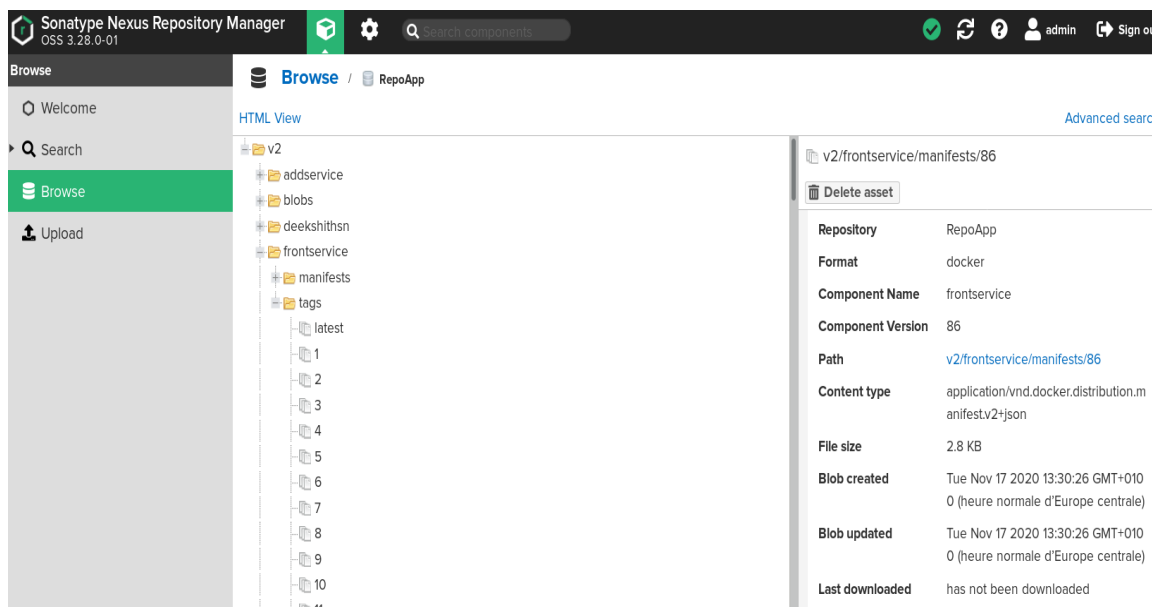


Figure 30 : Liste des images pousser dans nexus

L'étape suivante consiste à taguer et pousser l'image Docker vers un registry Docker (Nexus) (Figure 30).

```
[Pipeline] ansiblePlaybook
[frontservice] $ /usr/bin/ansible-playbook ansible.yaml -i hosts -b --become-user root

PLAY [test] *****

TASK [Gathering Facts] *****
ok: [192.168.30.136]

TASK [copy deployment file to kubernetes master] *****
ok: [192.168.30.136]

TASK [delete the previous of kubernetes objects] *****
changed: [192.168.30.136]

TASK [Create a Deployment by reading the definition from a local file] *****
changed: [192.168.30.136]

PLAY RECAP *****
192.168.30.136      : ok=4    changed=2    unreachable=0    failed=0
```

Figure 31 : Résultat d'exécution de Playbook d'ansible

Ensuite, l'Ansible exécute les Playbook et procède au déploiement dans le serveur Kubernetes (Figure 31).

NAME	READY	STATUS	RESTARTS	AGE
addservice-5b5b6c6cf4-n7v5h	1/1	Running	8	83d
devops-training-84dc6c7b8f-nlg6l	1/1	Running	6	82d
frontservice-bcf9547b6-j7dvw	1/1	Running	1	63d
microservice-55d7b7cb4f-kf7ff	1/1	Running	9	83d
nginx-6799fc88d8-5nq85	1/1	Running	29	94d
subservice-5d9bb6f54c-6bgb6	1/1	Running	8	83d

Figure 32 : Affichage des pod avec Kubernetes

En finale cette étape par l'exécution de notre application dans le serveur Kubernetes qui s'occupera de l'orchestration des pods (figure 32).

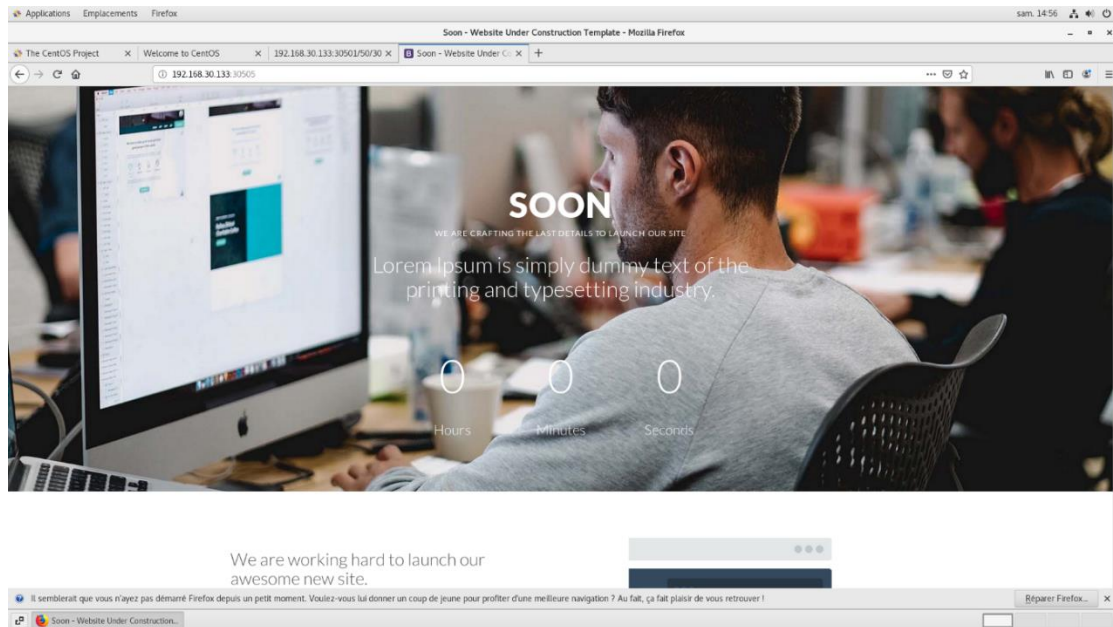


Figure 33 : L'interface de l'application déployée

Nous pouvons constater dans la figure 33 que l'interface expose l'application déployée sur le conteneur récemment créé.

3. Supervision des conteneurs

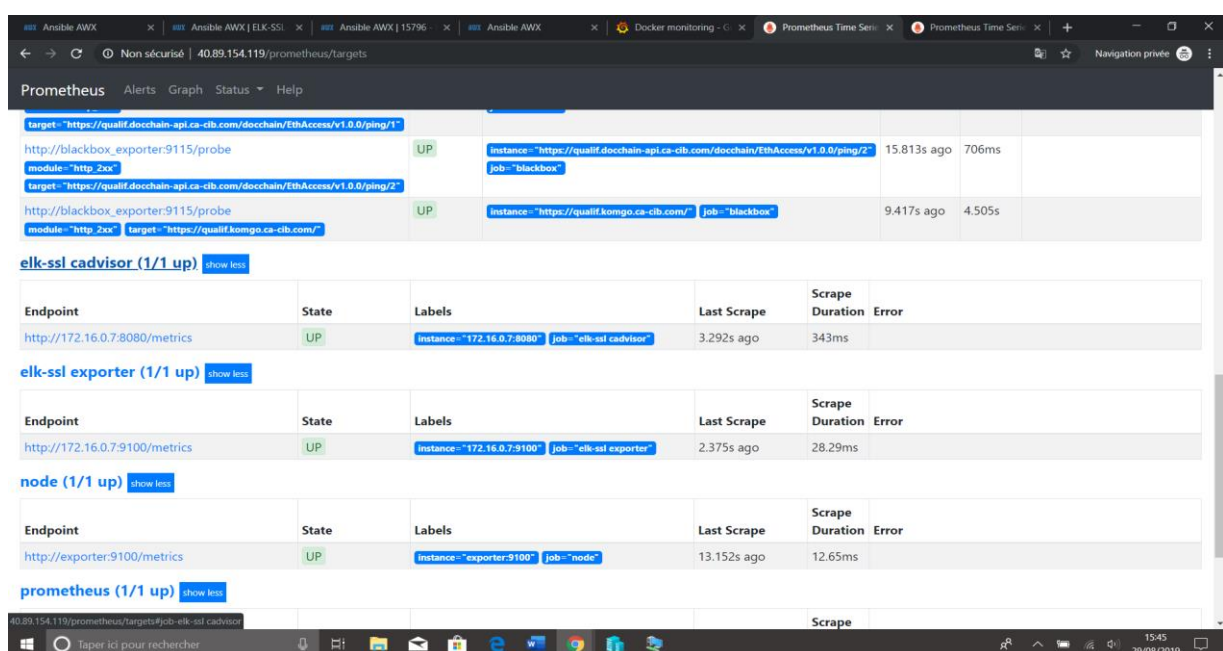


Figure 34 : Prometheus Target

La figure 34 illustre les cibles de Prometheus où il doit récolter les métriques et l'état des différents exporters. Les métriques sont stockées dans une base de données et interrogées à travers des API (HTTP) pour les récupérer. Dans notre cas on récupère les métriques propres aux machines, sur lesquelles nous avons déployé notre cluster Kubernetes et les autres conteneurs grâce à un Node exporter.

Bien que Prometheus nous permet de récolter les données des différentes machines néanmoins il n'offre pas une interface graphique de visualisations qui répond à nos besoins. Par conséquence et pour résoudre ce problème nous avons opté pour utiliser Grafana pour mettre en place des Dashboard plus dynamiques qui permettent de visualiser les métriques de nos machines.

En effet, nous avons trois Dashboard de visualisation de différentes métriques :

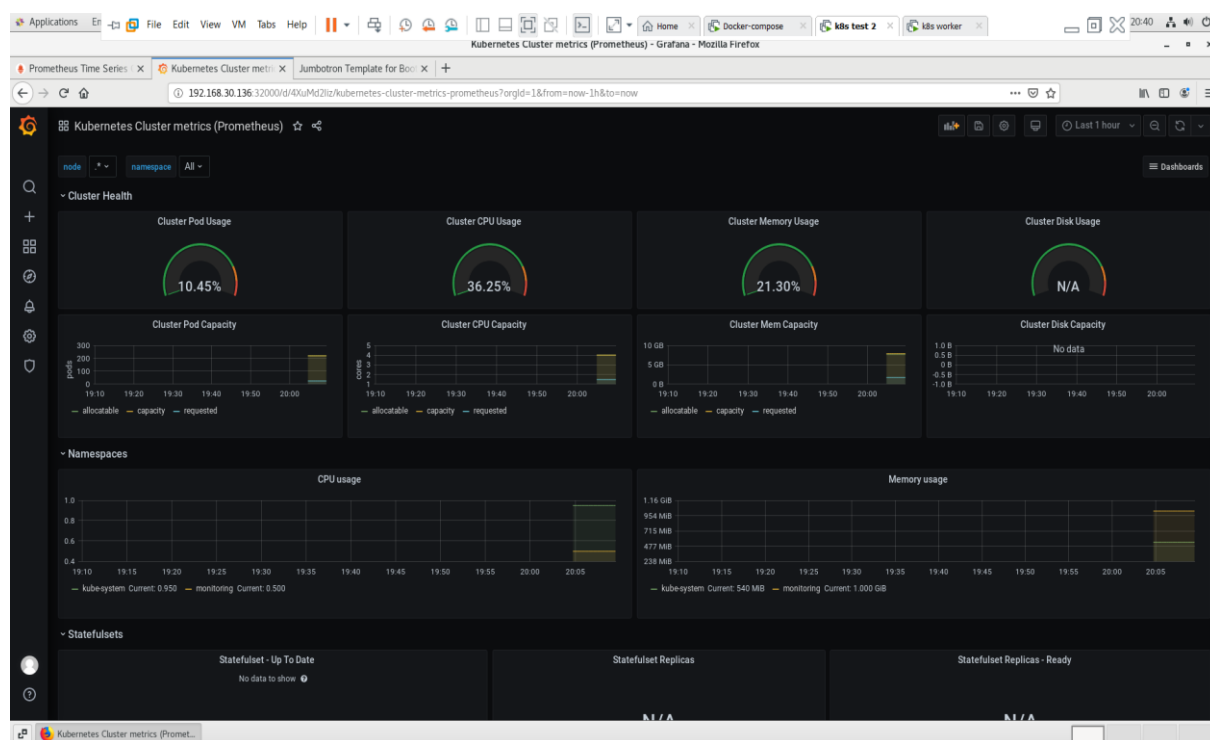


Figure 35 : Dashboard Kubernetes cluster metrics

Cette figure présente une Dashboard qui affiche les métriques du cluster Kubernetes en utilisant de différents graphes.

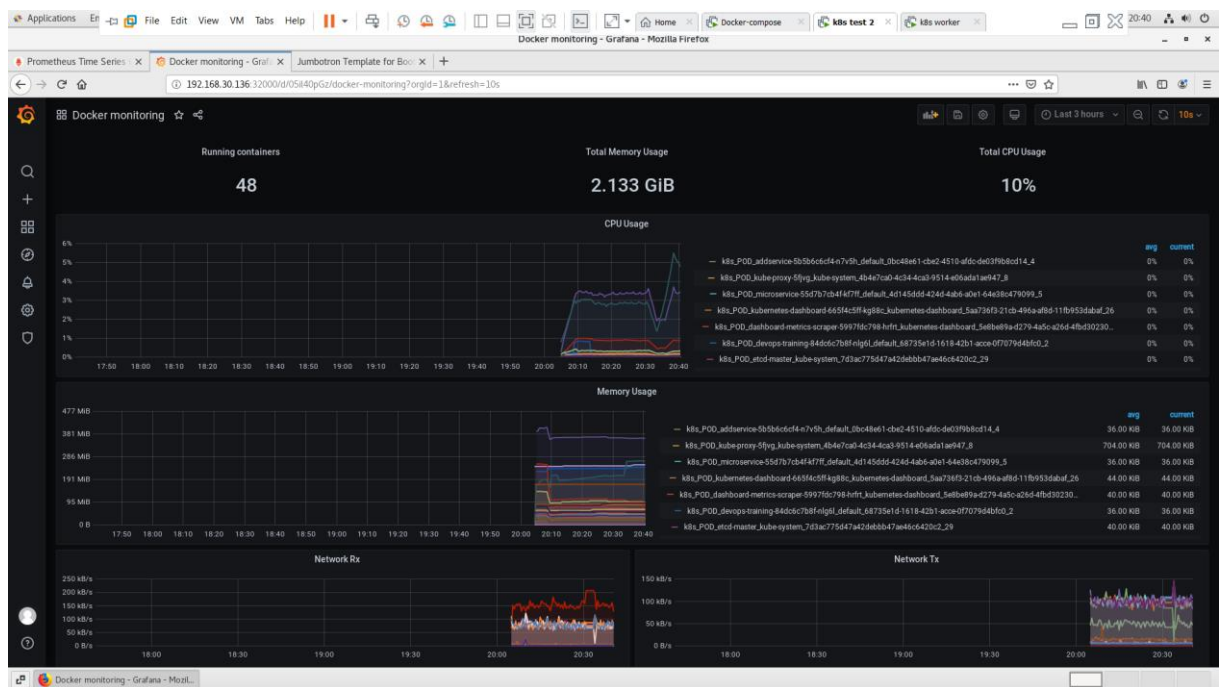


Figure 36 : Dashboard Docker metrics

Cette figure présente une Dashboard qui affiche les métriques des conteneurs Docker en utilisant de différents graphes.

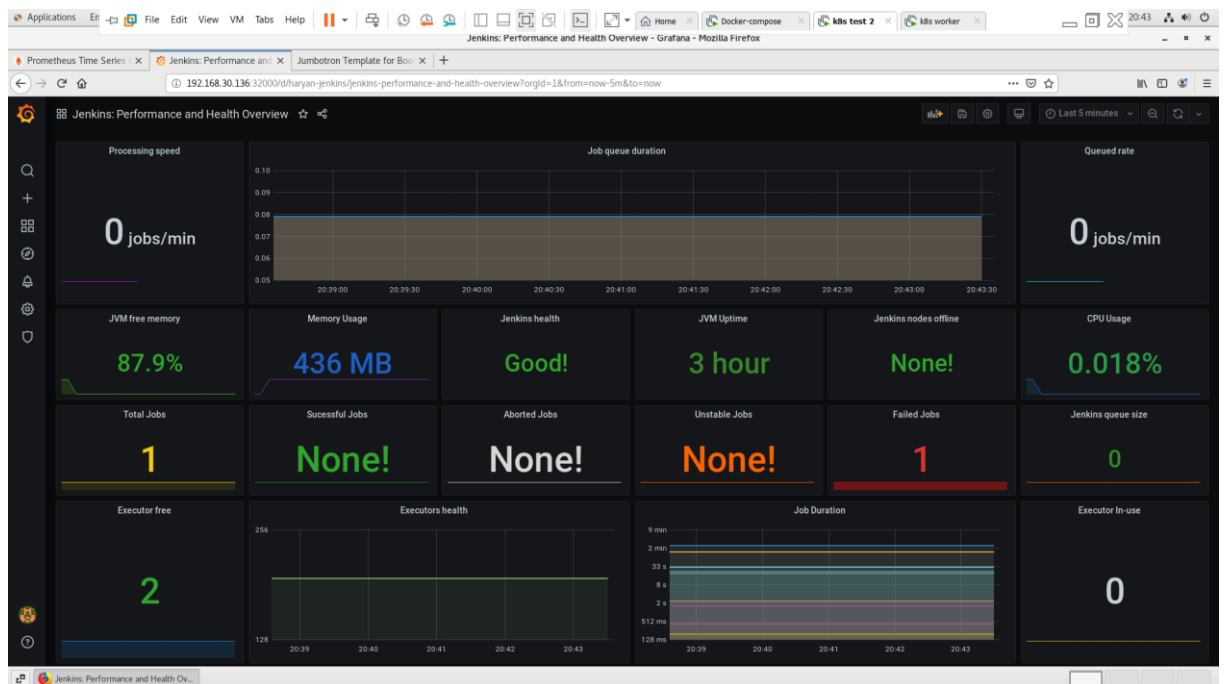


Figure 37 : Dashboard Jenkins Performance

Cette figure illustre le tableau de bords qui affiche les métriques du Jenkins en utilisant de différents graphes.

Conclusion

Dans ce chapitre nous avons entamé la réalisation du projet tout en insistant sur quelques scénarios qui montrent les fonctionnalités majeures de l'application.

Conclusion générale et perspectives

Dans le cadre de la préparation du projet de fin d'études, nous avons créé une plateforme DevOps au sein de la société Studi. Notre travail consiste à automatiser le processus de la création d'un projet en l'implémentant dans toute une chaîne d'intégration et de déploiement continu, à base de docker, en utilisant un système de surveillance.

Afin de bien présenter notre projet, nous avons décidé de subdiviser le travail en quatre chapitres. Le premier chapitre est une présentation du cadre général du projet, où nous avons consacré une partie pour présenter l'organisme d'accueil, entamer l'étude de l'existant, cerner les objectifs et enfin appliquer la méthodologie adoptée. Le deuxième chapitre consiste à présenter l'étude théorique, où nous avons en premier définis les différents concepts de base et par la suite menée une étude comparative des technologies adoptées. Après avoir bien définies les technologies adoptées dans le cadre de ce projet, nous sommes passé dans le troisième chapitre à l'analyse des besoins et la conception. Enfin, nous avons abordé la partie réalisation où nous avons présenté l'environnement du travail et les principaux développements.

Nous avons atteint nos objectifs, à la fin de cette période de ce stage, en créant une plateforme conforme aux besoins étudiés et en respectant les contraintes techniques imposées.

Comme toute recherche scientifique nous avons rencontré des difficultés, notamment dans la compréhension de l'architecture complexe que nous avons implémentée. Toutefois, nous avons réussi à les surmonter, ce qui nous a permis de maîtriser des outils innovants et renforcer nos connaissances pour les outils acquis au cours de notre formation.

Nous aimons à la fin souligner quelques extensions intéressantes pour améliorer le travail, en effet, nous estimons que la migration de notre architecture vers le cloud est souhaitable pour renforcer la résilience et assurer la haute disponibilité des services.

Au terme de ce travail, nous souhaitons que ce projet de fin d'études soit aux attentes des encadrants de l'entreprise hôte STUDI et aux membres du Jury.

Webographie

- [1] <https://bubbleplan.net/blog/agile-scrum-gestion-projet/> Consulté le: 02/06/2020
- [2] <https://devopssec.fr/article/differences-virtualisation-et-conteneurisation> Consulté le:
30/08/2020
- [3] http://devopsenterprise.io/media/DOES_forum_metrics_102015.pdf Consulté le:
15/11/2020
- [4] <https://www.padok.fr/en/blog/monitoring-tools-kubernetes> Consulté le: 30/11/2020
- [5] <https://www.lebigdata.fr/kubernetes-definition> Consulté le: 30/11/2020
- [6] <https://www.ionos.fr/digitalguide/sites-internet/developpement-web/gitlab-vs-github/>
Consulté le : 20/12/2020

Liste des acronymes

VM : Machine virtuelle

CI/DI : Continues Integration / Deployment Integration

K8S : Kubernetes

API : Application Programming Interface

HTTP : HyperText Markup Transfer Protocol

URL : Uniform Resource Locator

SSH : Secure Shell

Dev : les équipes chargées du développement

Ops : l'équipe opérationnelle chargée de l'exploitation des systèmes

IAAC : Infrastructure as a code

GUI : Graphical User Interface

SLA : Service Level Agreement

OS : système d'exploitation

CPU : Central Processing Unit

RAM : Random Access Memory

Résumé

Le présent travail s'inscrit dans le cadre de la réalisation d'un projet de fin d'étude afin d'obtenir un diplôme national d'ingénieur, spécialité informatique de l'école supérieure privée d'ingénierie et de technologies.

Notre projet consiste à mettre en place une plateforme DevOps qui permettra la génération de projets sous une chaîne d'intégration et de déploiement continu.

Abstract

This work is part of my graduation project for the purpose to obtain a national Engineering Diploma, in computer science specialty from the private higher school of Engineering and Technology.

Our project consists on setting up a DevOps platform that requires the generation of projects under a chain of continuous integration and deployment.

Urkund Analysis Result

Analysed Document: rapport touj (3).docx (D94359525)
Submitted: 2/1/2021 11:23:00 PM
Submitted By: anouar.chemek@esprit.tn
Significance: 7 %

Sources included in the report:

RapportPFE_AmineZouai_Finale.docx (D41375153)

Instances where selected sources appear:

30



ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454